

Non-Parametric Localization

Grid and Monte-Carlo Localization

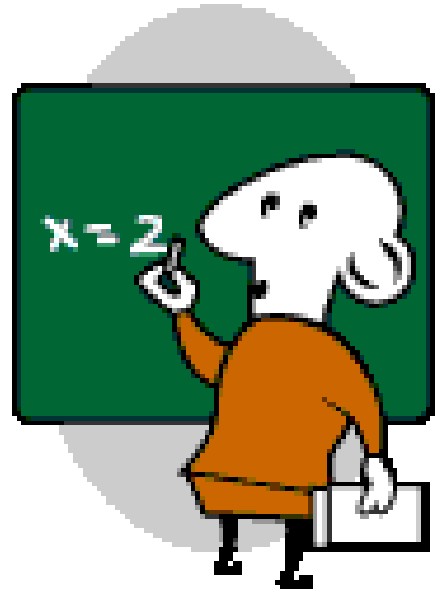
Department of Electrical and Electronics Engineering
Dr. Afşar Saranlı

*Lecture slides heavily use material from the textbook and
Sebastian Thrun, Lecture Slides; <http://www.probabilistic-robotics.org/>*



What we will discuss

- We will concentrate on application, practical issues and examples,
- Discuss Grid Localization,
- Practical issues and examples,
- Discuss Monte-Carlo Localization (MCL),
- Practical Issues and Examples,
- Recovery from Failures (Kidnapped Robot),
- Handling dynamic environments.





Properties Revisited

- Non-parametric localization has nice features that Gaussian techniques lack:
- Very easy to implement,
- Can process raw sensor measurements,
- Being non-parametric, they can represent arbitrary posterior densities, hence:
- Can solve global localization problems, including kidnapped robot problem (to some extend),
- Standard Gaussian filters (KF, EKF) cannot solve these problems.



Grid Localization

- Uses the Histogram Filter to represent the robot state posterior,
- It is an instance of the Discrete Bayes Filter:

```
1:   Algorithm Grid_localization( $\{p_{k,t-1}\}, u_t, z_t, m$ ):  
2:     for all  $k$  do  
3:        $\bar{p}_{k,t} = \sum_i p_{i,t-1} \text{motion\_model}(\text{mean}(\mathbf{x}_k), u_t, \text{mean}(\mathbf{x}_i))$   
4:        $p_{k,t} = \eta \text{measurement\_model}(z_t, \text{mean}(\mathbf{x}_k), m)$   
5:     endfor  
6:     return  $\{p_{k,t}\}$ 
```

- Grid resolution is a major factor for consideration.



Grid Localization

Posterior represented by a discrete set of probabilities: one for each grid cell.

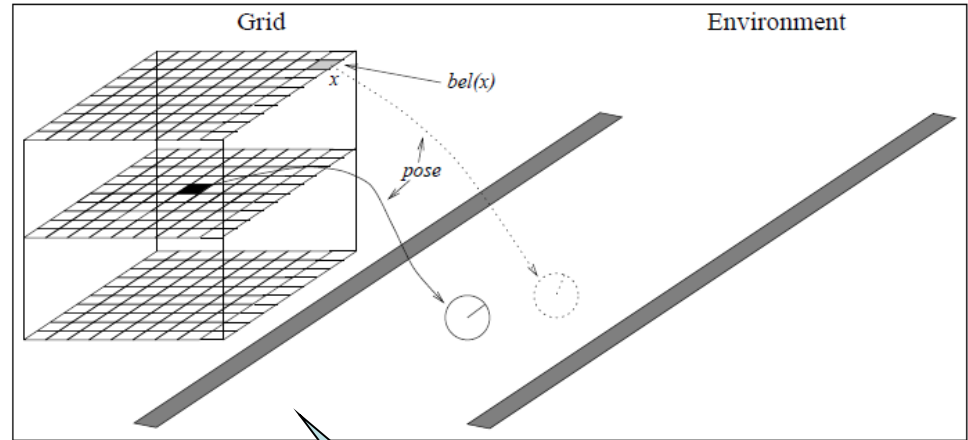
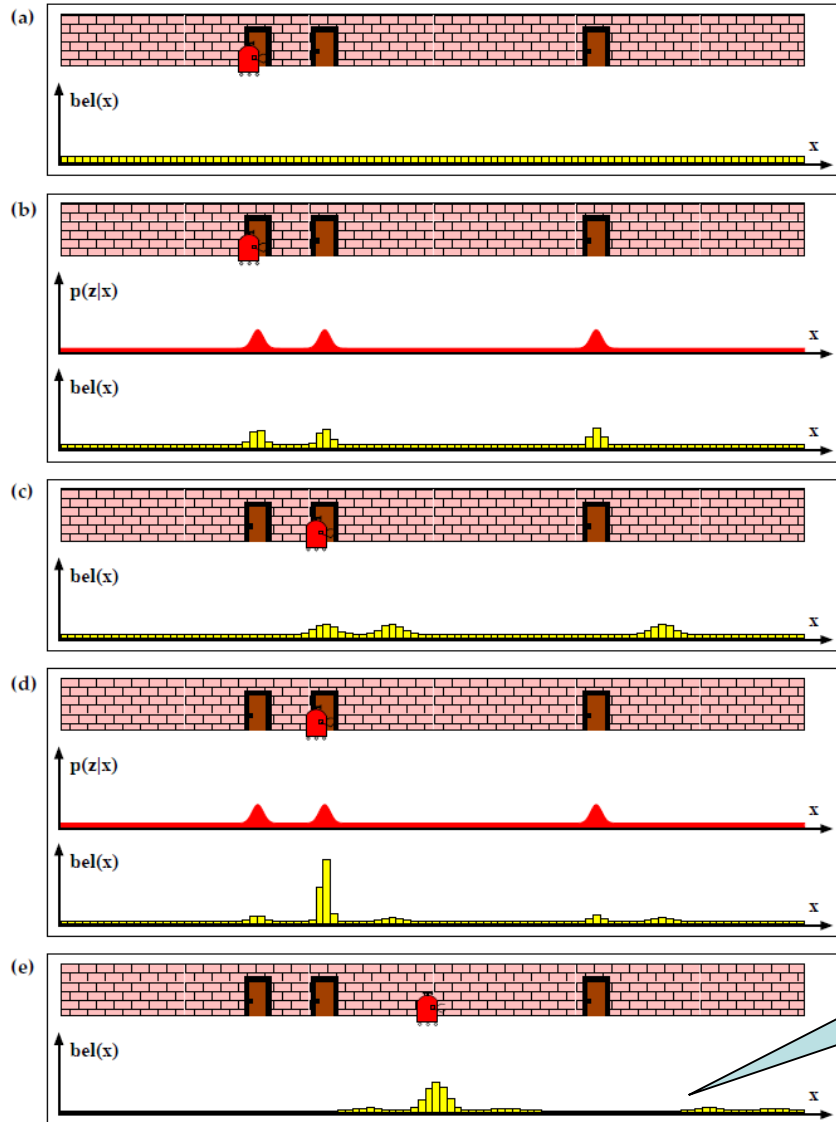
```
1:  Algorithm Grid_localization( $\{p_{k,t-1}\}, u_t, z_t, m$ ):  
2:    for all  $k$  do  
3:       $\bar{p}_{k,t} = \sum_i p_{i,t-1} \text{motion\_model}(\text{mean}(\mathbf{x}_k), u_t, \text{mean}(\mathbf{x}_i))$   
4:       $p_{k,t} = \eta \text{measurement\_model}(z_t, \text{mean}(\mathbf{x}_k), m)$   
5:    endfor  
6:    return  $\{p_{k,t}\}$ 
```

One of the models developed in earlier lectures.

Mean(.) function computes “Center of mass” of the grid cell k in the state space



Uniform Grid (Metric) Example



2D robot pose
example

1D
hallway
example



Uniform Grid (Metric) Issues

- A fine grid gives better resolution but at the cost of computational complexity,
- Coarse grid may be chosen to reduce complexity at the expense of reduced resolution,
- Also: coarse grids may need modification of *motion models* and *sensor models* to take into account the “grid noise”
- One fix: Increase the noise variance of sensor and motion model,
- There are other techniques as well.

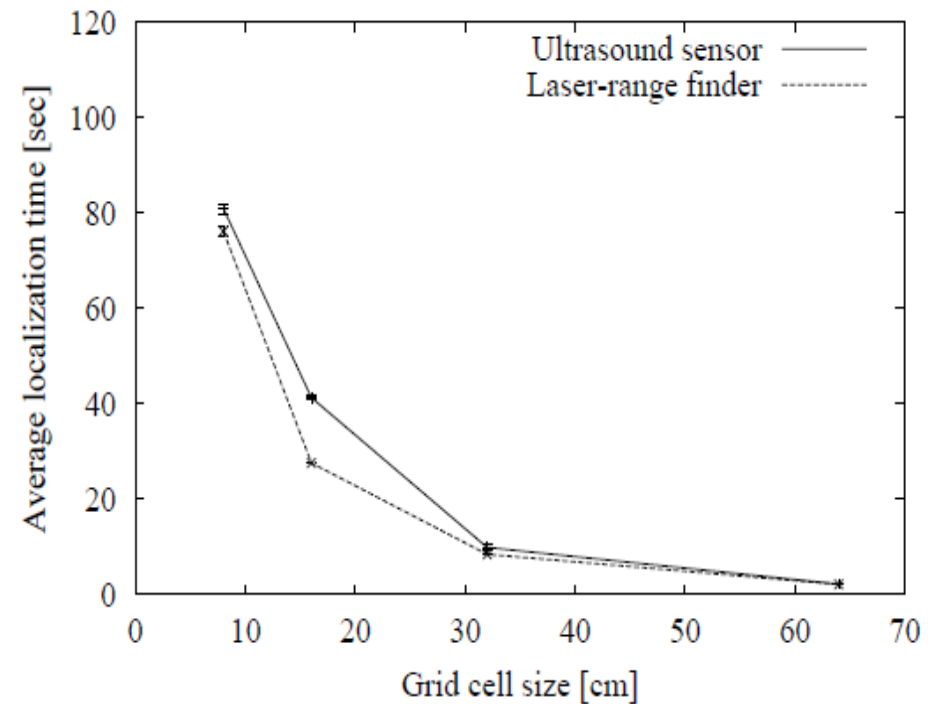
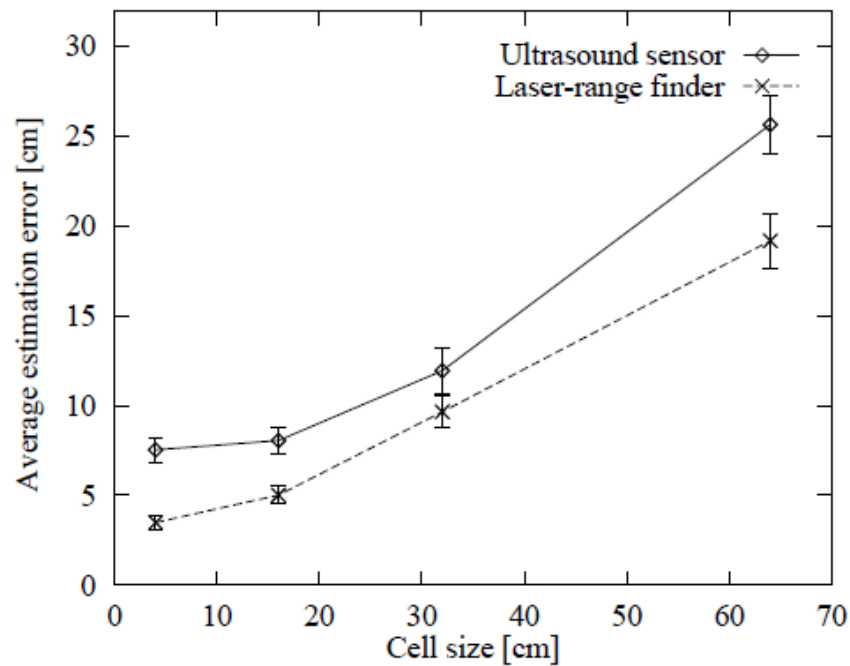


Computational Considerations

- Uniform grid localization is computationally expensive in the practical case,
- ***Model pre-caching***: Aimed at speeding-up an expensive measurement model for a fixed map. (e.g. correct range for each grid cell in ray casting)
- ***Sensor Sub-Sampling***: Select a sub-set of all range measurements, e.g., 8 instead of 360 measurements in a laser scanner,
- ***Delayed motion Updates***: Lower frequency updates for motion model as compared to measurement freq. of robot,
- ***Selective Updating***: Update only grid cells with high posterior likelihood



Uniform Grid (Metric) Performance





Topological Grid Example

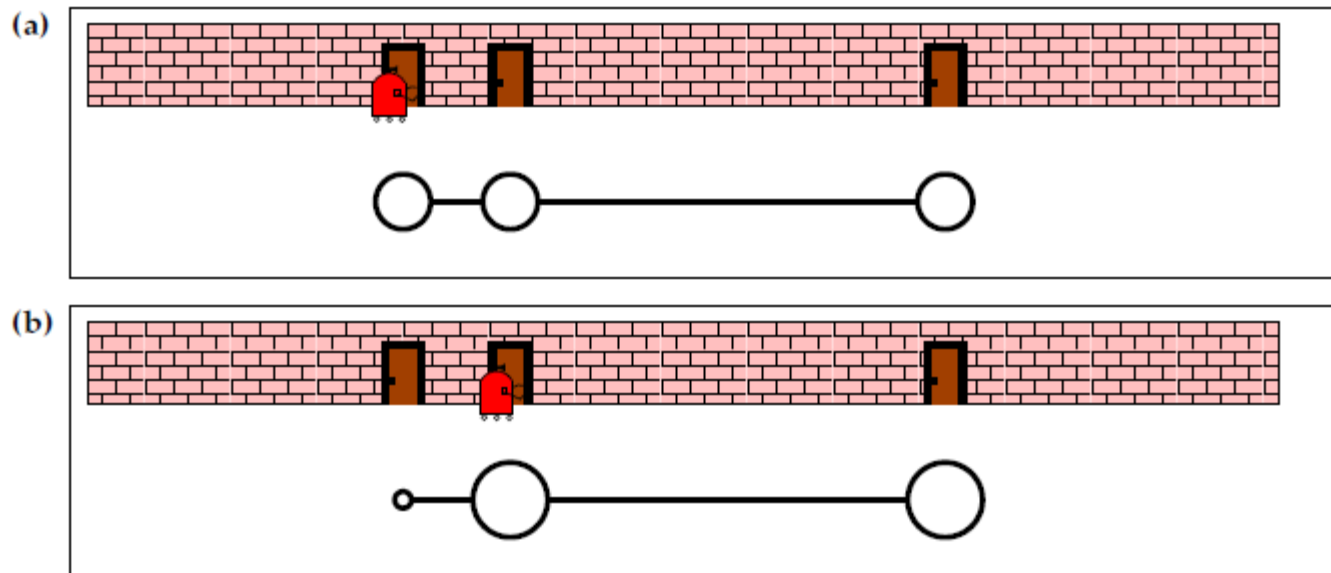
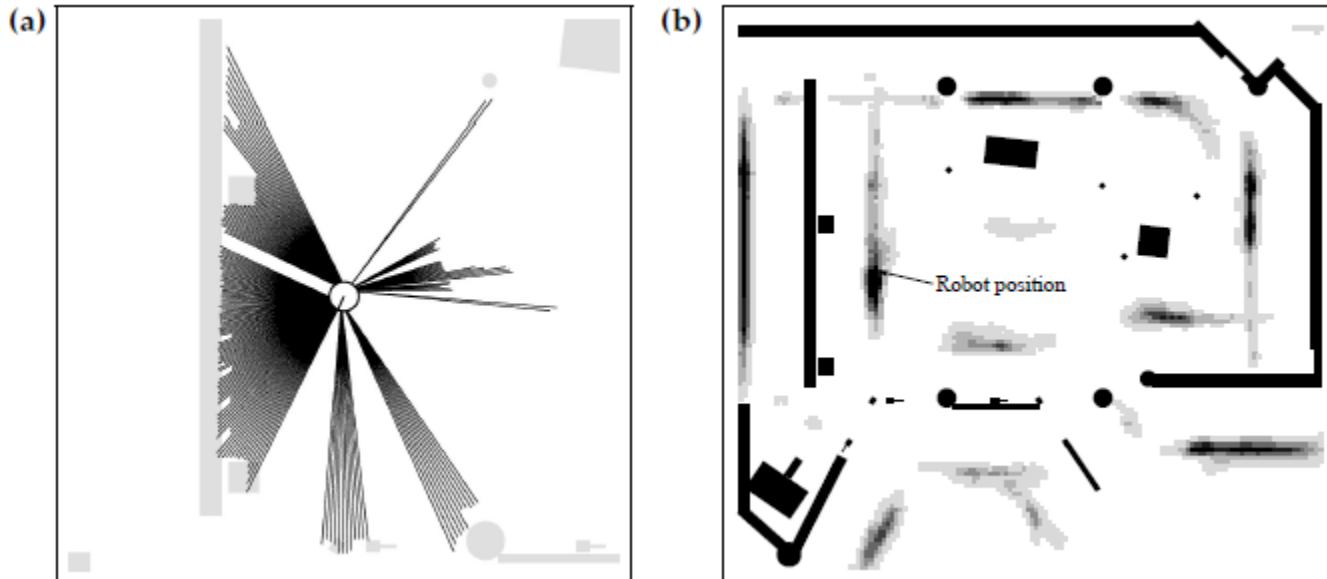


Figure 8.5 Application of a coarse-grained, topological representation to mobile robot localization. Each state corresponds to a distinctive place in the environment (a door in this case). The robot's belief $bel(x_t)$ of being in a state is represented by the size of the circles. (a) The initial belief is uniform over all poses. (b) shows the belief after the robot made one state transition and detected a door. At this point, it is unlikely that the robot is still in the left position.



Grid Localization Example

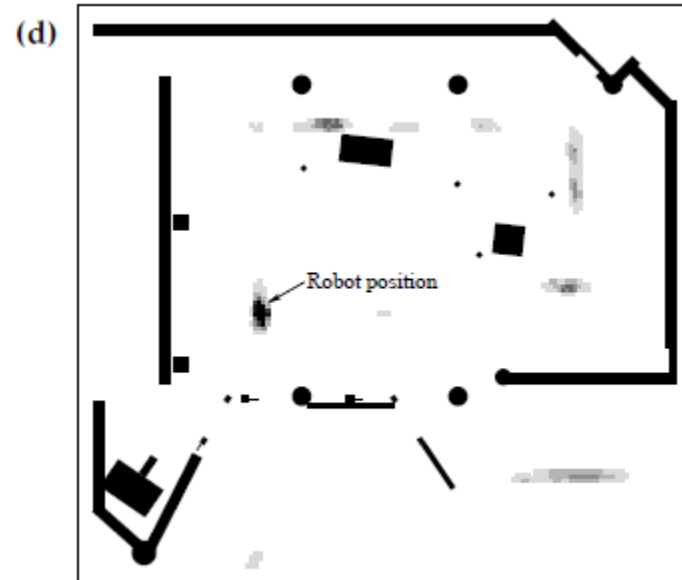
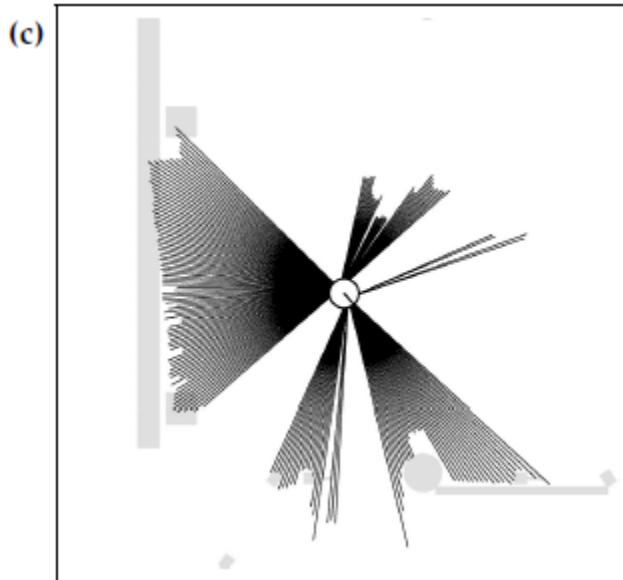
- Uniform grid global localization. **Two laser scanners**, Grid size: 15cmx15cm position, 5 deg orientation (typical for indoor scenario).





Grid Localization Example

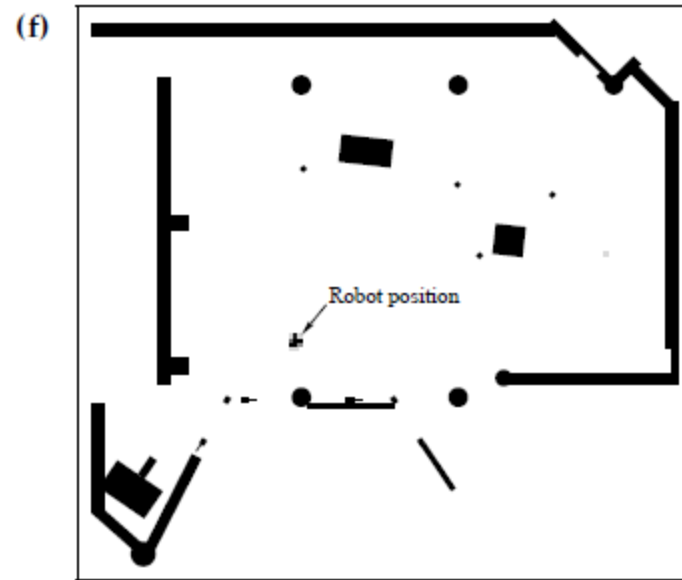
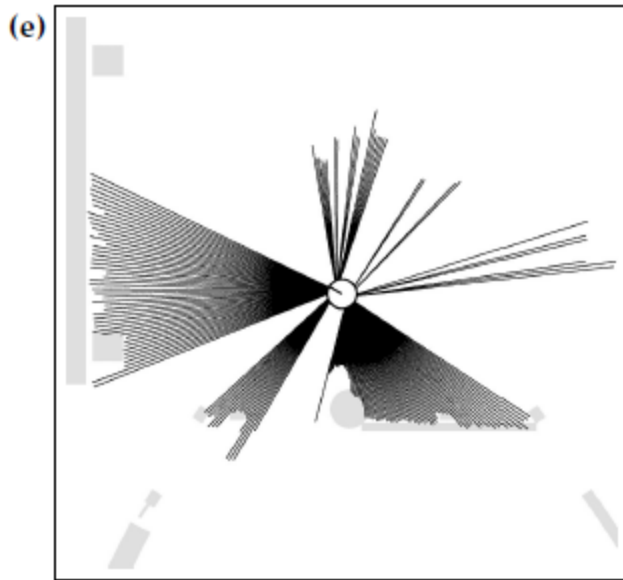
- Uniform grid global localization. **Two laser scanners**, Grid size: 15cmx15cm position, 5 deg orientation (typical for indoor scenario).





Grid Localization Example

- Uniform grid global localization. **Two laser scanners**, Grid size: 15cmx15cm position, 5 deg orientation (typical for indoor scenario).

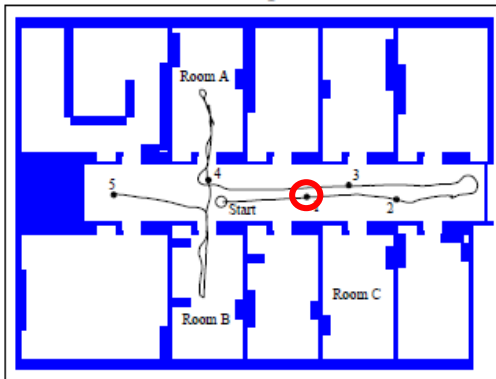




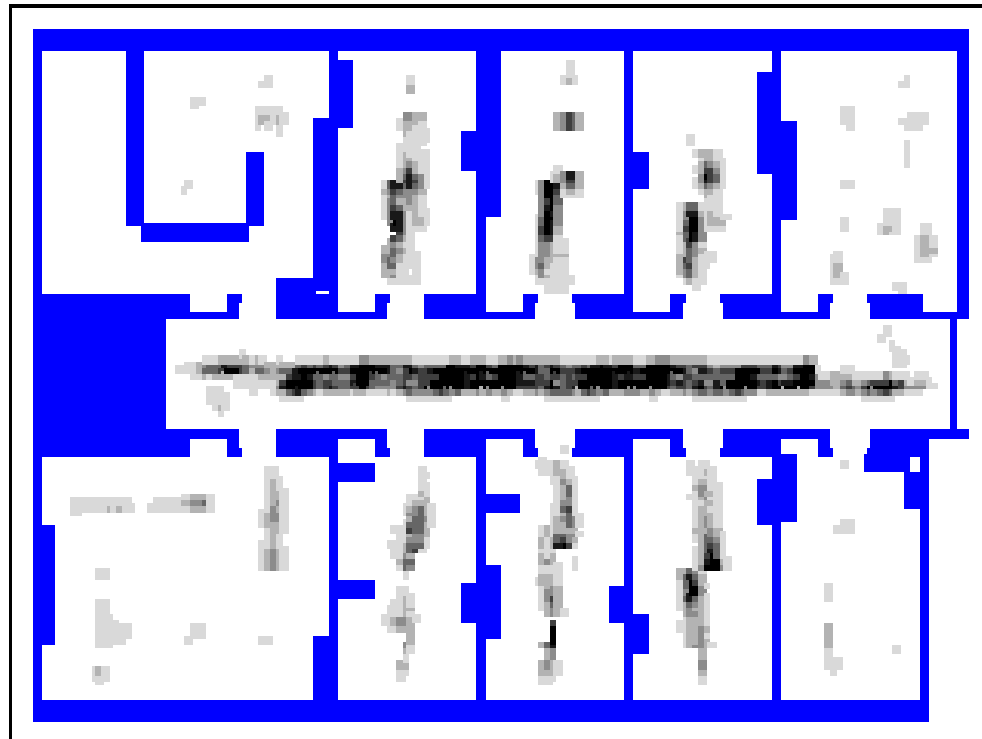
Grid Localization Example

- Uniform grid global localization. **Sonar Range Scanner**, Grid size: 15cmx15cm position, 5 deg orientation (typical for indoor scenario).

(a) Path and reference poses



(b) Belief at reference pose 1

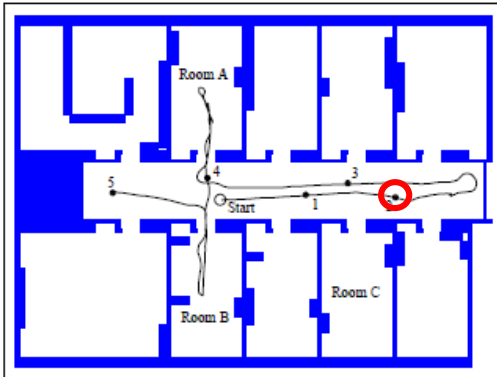




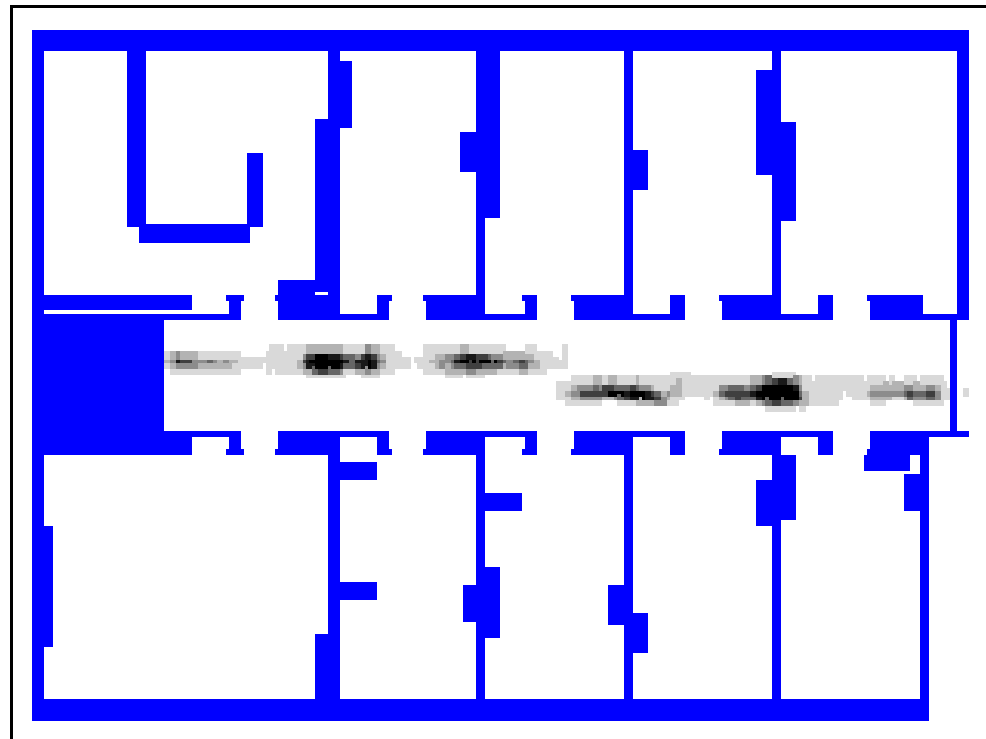
Grid Localization Example

- Uniform grid global localization. **Sonar Range Scanner**, Grid size: 15cmx15cm position, 5 deg orientation.

(a) Path and reference poses



(c) Belief at reference pose 2

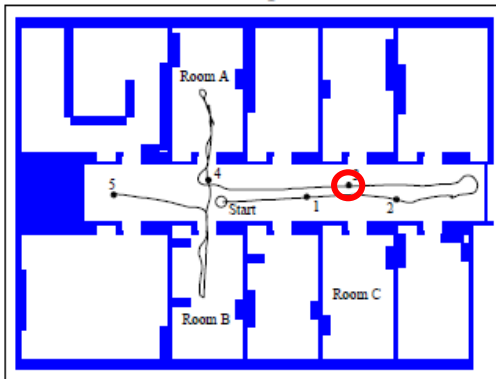




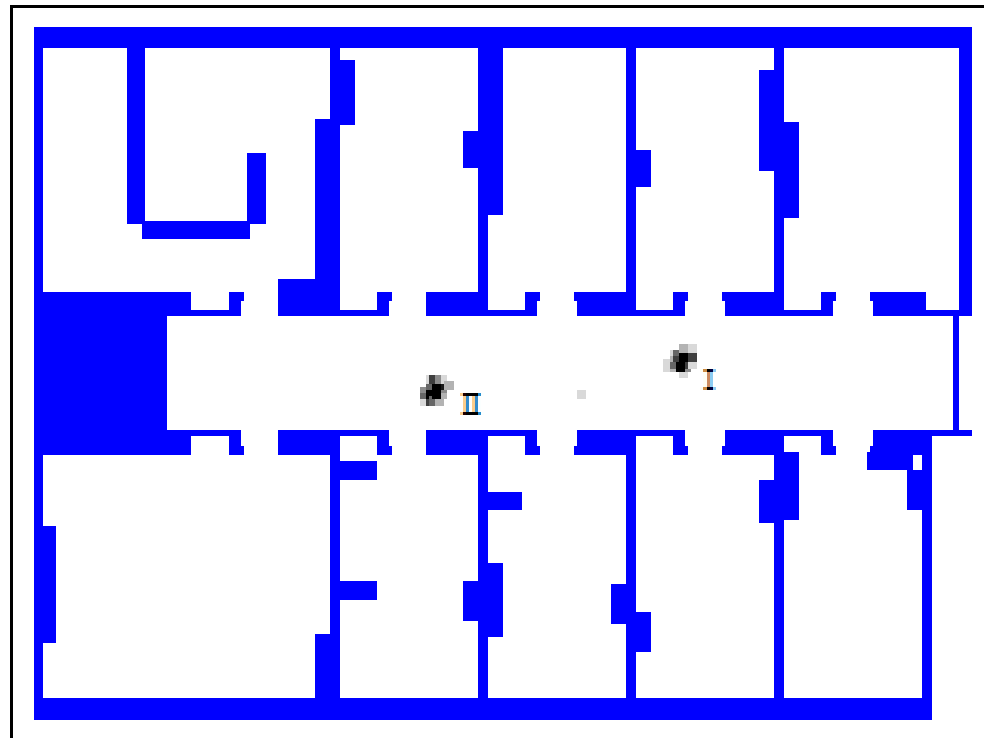
Grid Localization Example

- Uniform grid global localization. **Sonar Range Scanner**, Grid size: 15cmx15cm position, 5 deg orientation.

(a) Path and reference poses



(d) Belief at reference pose 3

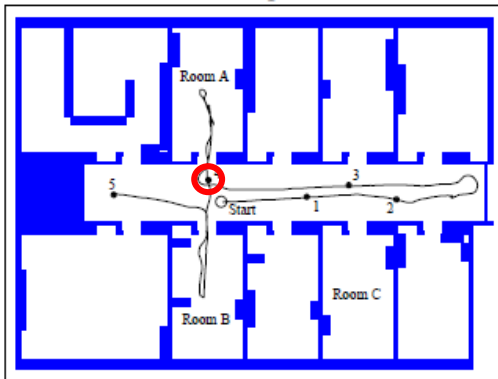




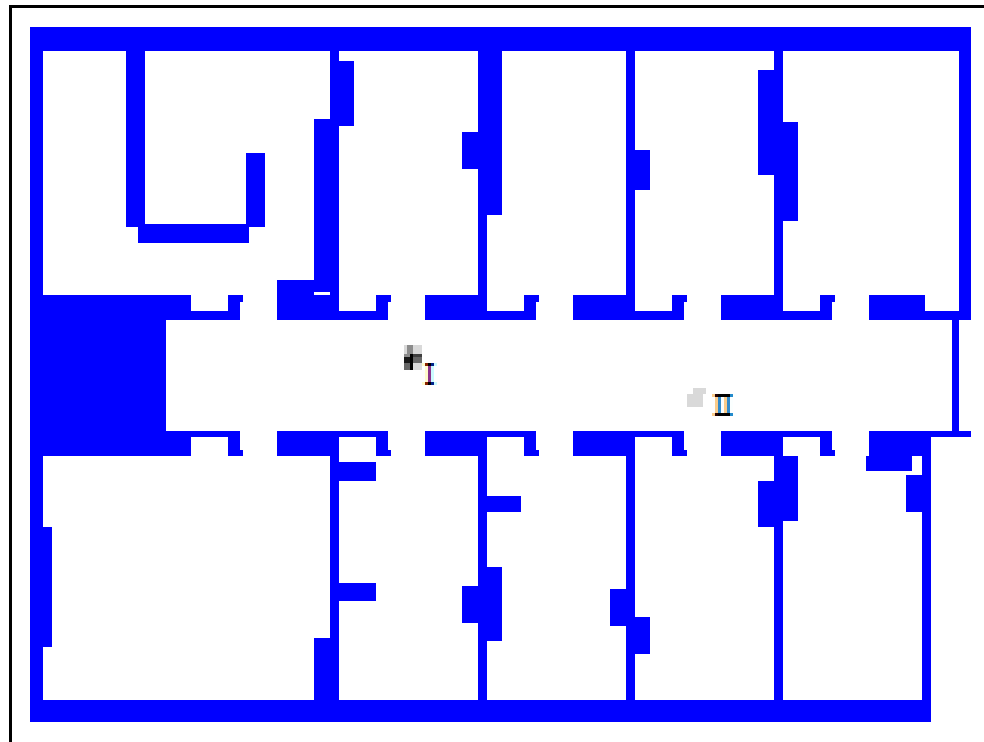
Grid Localization Example

- Uniform grid global localization. **Sonar Range Scanner**, Grid size: 15cmx15cm position, 5 deg orientation.

(a) Path and reference poses



(e) Belief at reference pose 4

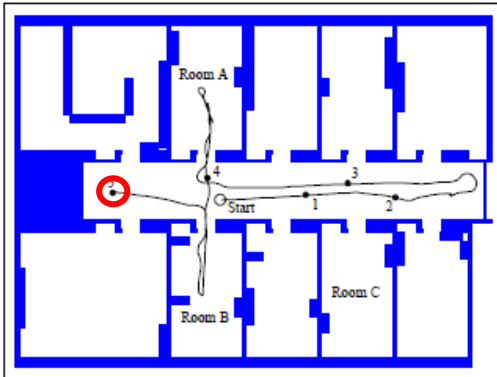




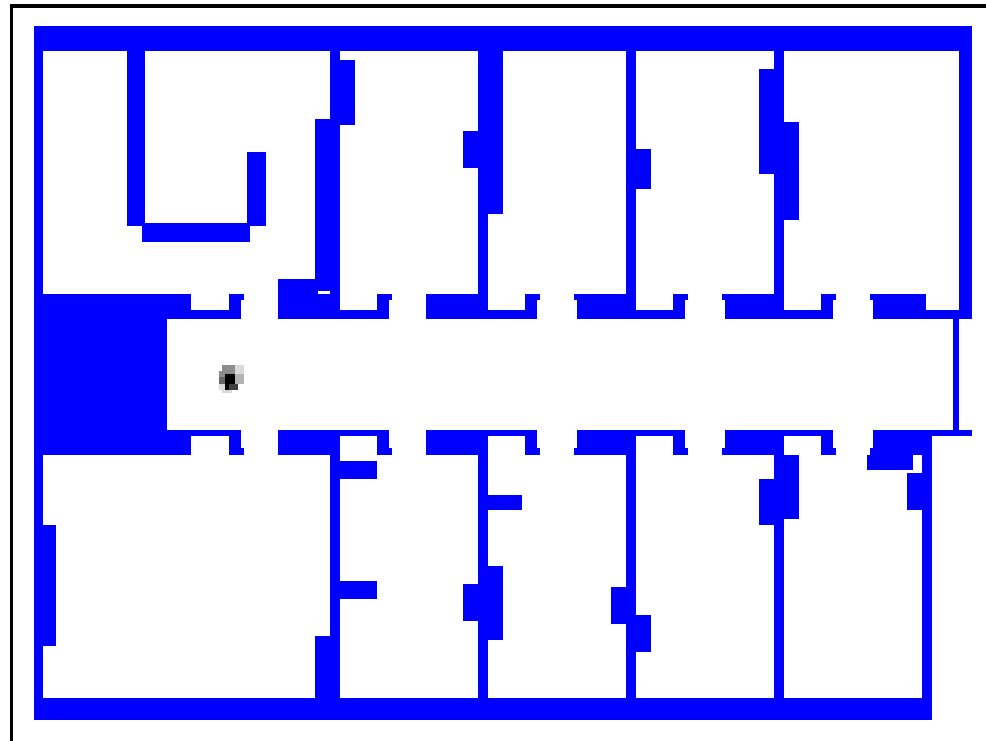
Grid Localization Example

- Uniform grid global localization. **Sonar Range Scanner**, Grid size: 15cmx15cm position, 5 deg orientation.

(a) Path and reference poses



(f) Belief at reference pose 5 (after visiting Room B)





Grid Localization Example

- Simple odometry versus localization

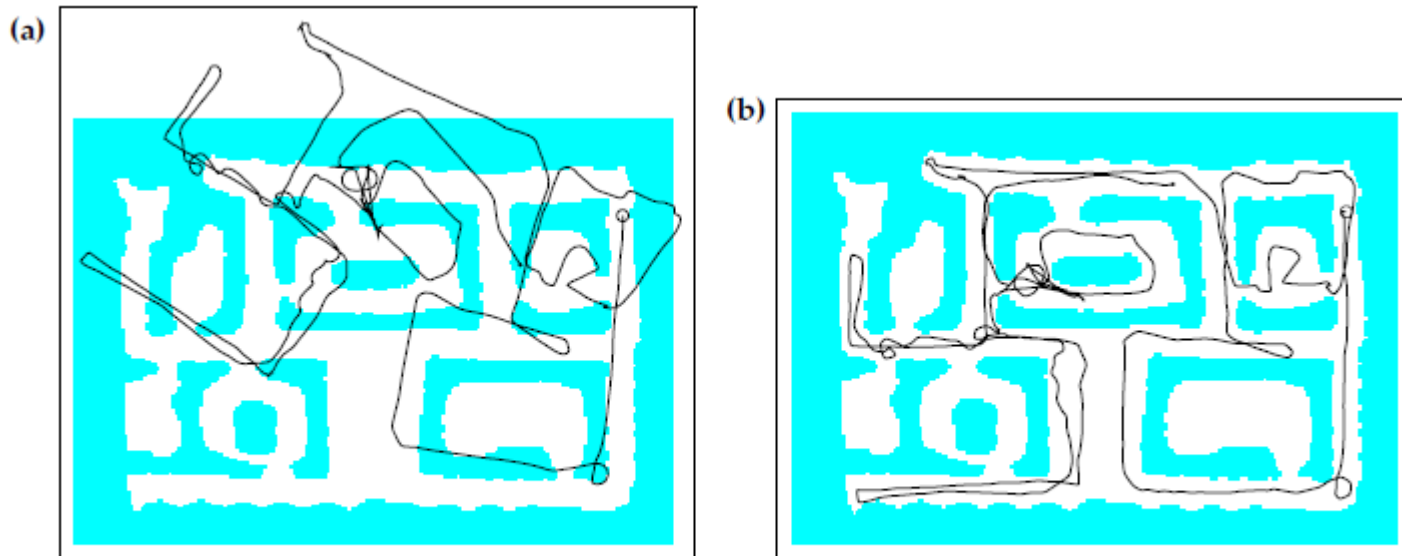


Figure 8.10 (a) Odometry information and (b) corrected path of the robot.



Monte-Carlo Localization

- Uses the Particle Filter to represent the robot state posterior,

```
1: Algorithm MCL( $\mathcal{X}_{t-1}, u_t, z_t, m$ ):  
2:    $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$   
3:   for  $m = 1$  to  $M$  do  
4:      $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$   
5:      $w_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$   
6:      $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$   
7:   endfor  
8:   for  $m = 1$  to  $M$  do  
9:     draw  $i$  with probability  $\propto w_t^{[i]}$   
10:    add  $x_t^{[i]}$  to  $\mathcal{X}_t$   
11:  endfor  
12:  return  $\mathcal{X}_t$ 
```

```
1: Algorithm Particle_filter( $\mathcal{X}_{t-1}, u_t, z_t$ ):  
2:    $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$   
3:   for  $m = 1$  to  $M$  do  
4:     sample  $x_t^{[m]} \sim p(x_t | u_t, x_{t-1}^{[m]})$   
5:      $w_t^{[m]} = p(z_t | x_t^{[m]})$   
6:      $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$   
7:   endfor  
8:   for  $m = 1$  to  $M$  do  
9:     draw  $i$  with probability  $\propto w_t^{[i]}$   
10:    add  $x_t^{[i]}$  to  $\mathcal{X}_t$   
11:  endfor  
12:  return  $\mathcal{X}_t$ 
```

Main difference
is the presence
of the map m

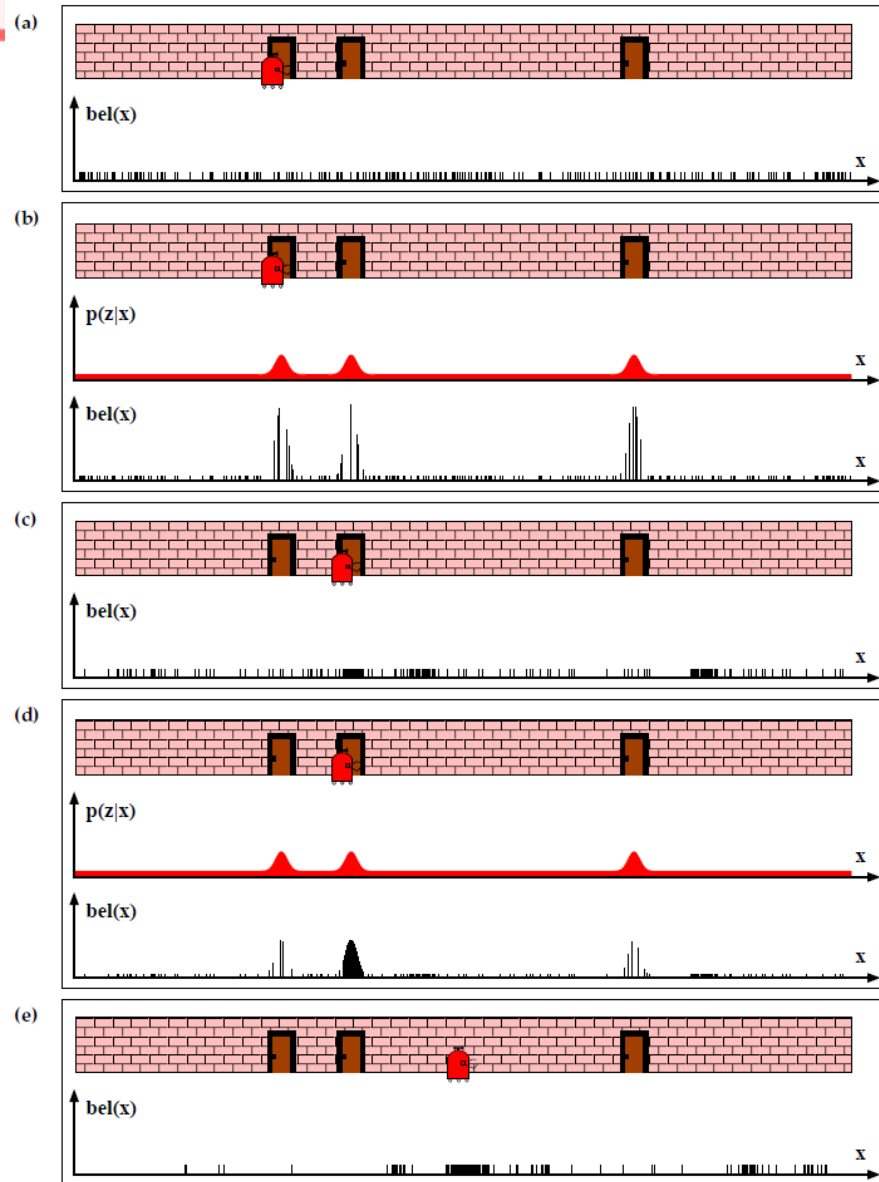


MCL Properties

- Relatively new,
- Very easy to implement,
- Can represent arbitrary posterior density functions about robot state, including complex multi-modal density functions,
- Can handle a large class of problems, including global localization and kidnapped robot problem.
- Number of Particles M is the means to trade-off accuracy with computational complexity,
- **Has become one of the most popular localization algorithms in robotics.**



MCL Example (1D localization)





Motion Model Reminder

```

1: Algorithm MCL( $\mathcal{X}_{t-1}, u_t, z_t, m$ ):
2:    $\tilde{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$ 
3:   for  $m = 1$  to  $M$  do
4:      $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$ 
5:      $w_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$ 
6:      $\tilde{\mathcal{X}}_t = \tilde{\mathcal{X}}_t + \{x_t^{[m]}, w_t^{[m]}\}$ 
7:   endfor
8:   for  $m = 1$  to  $M$  do
9:     draw  $i$  with probability  $\propto w_t^{[i]}$ 
10:    add  $x_t^{[i]}$  to  $\mathcal{X}_t$ 
11:   endfor
12:   return  $\mathcal{X}_t$ 

```

Algorithm **sample_motion_model**(u, x):

1. $\hat{\delta}_{rot1} = \delta_{rot1} + \text{sample}(\alpha_1 | \delta_{rot1} | + \alpha_2 \delta_{trans})$
2. $\hat{\delta}_{trans} = \delta_{trans} + \text{sample}(\alpha_3 \delta_{trans} + \alpha_4 (|\delta_{rot1}| + |\delta_{rot2}|))$
3. $\hat{\delta}_{rot2} = \delta_{rot2} + \text{sample}(\alpha_1 | \delta_{rot2} | + \alpha_2 \delta_{trans})$
4. $x' = x + \hat{\delta}_{trans} \cos(\theta + \hat{\delta}_{rot1})$
5. $y' = y + \hat{\delta}_{trans} \sin(\theta + \hat{\delta}_{rot1})$
6. $\theta' = \theta + \hat{\delta}_{rot1} + \hat{\delta}_{rot2}$
7. **Return** $\langle x', y', \theta' \rangle$

Generate "noisy controls"

Find "noisy next states"

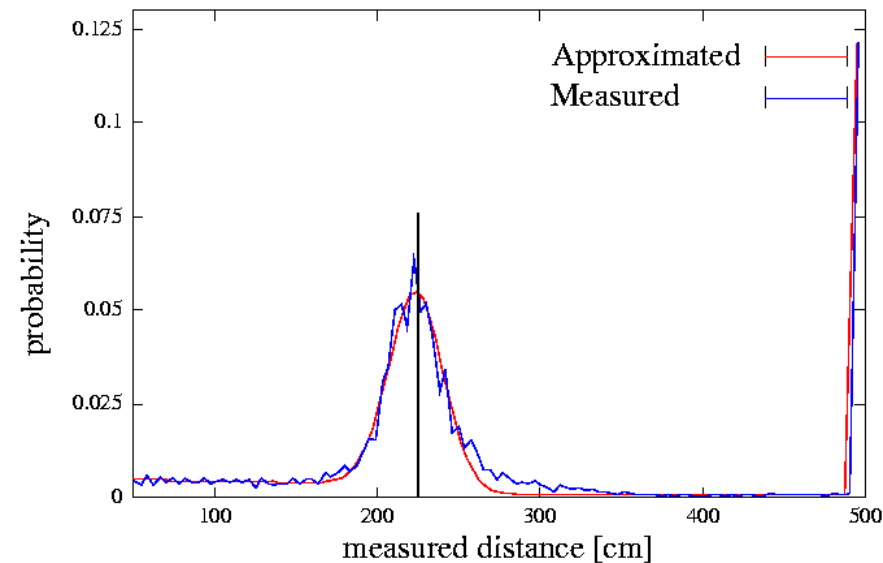
Example:
"sample_normal_distribution"

10 meters

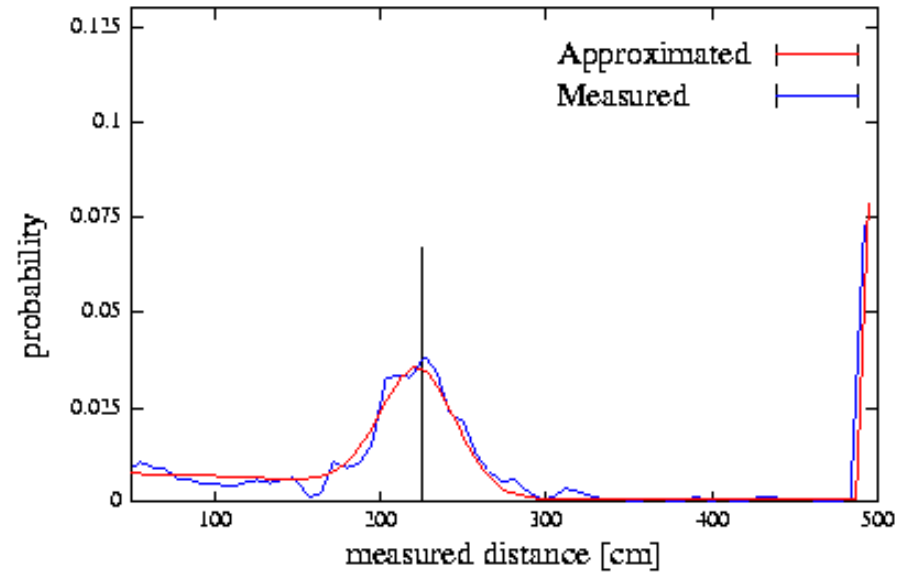


Range Sensor Model Reminder

```
1: Algorithm MCL( $\mathcal{X}_{t-1}, u_t, z_t, m$ ):  
2:    $\hat{\mathcal{X}}_t = \mathcal{X}_{t-1} = \emptyset$   
3:   for  $m = 1$  to  $M$  do  
4:      $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$   
5:      $z_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$   
6:      $\hat{\mathcal{X}}_t = \mathcal{X}_t + (x_t^{[m]}, w_t^{[m]})$   
7:   endfor  
8:   for  $m = 1$  to  $M$  do  
9:     draw  $i$  with probability  $\propto w_t^{[i]}$   
10:    add  $x_t^{[i]}$  to  $\mathcal{X}_t$   
11:   endfor  
12:   return  $\mathcal{X}_t$ 
```



Laser sensor

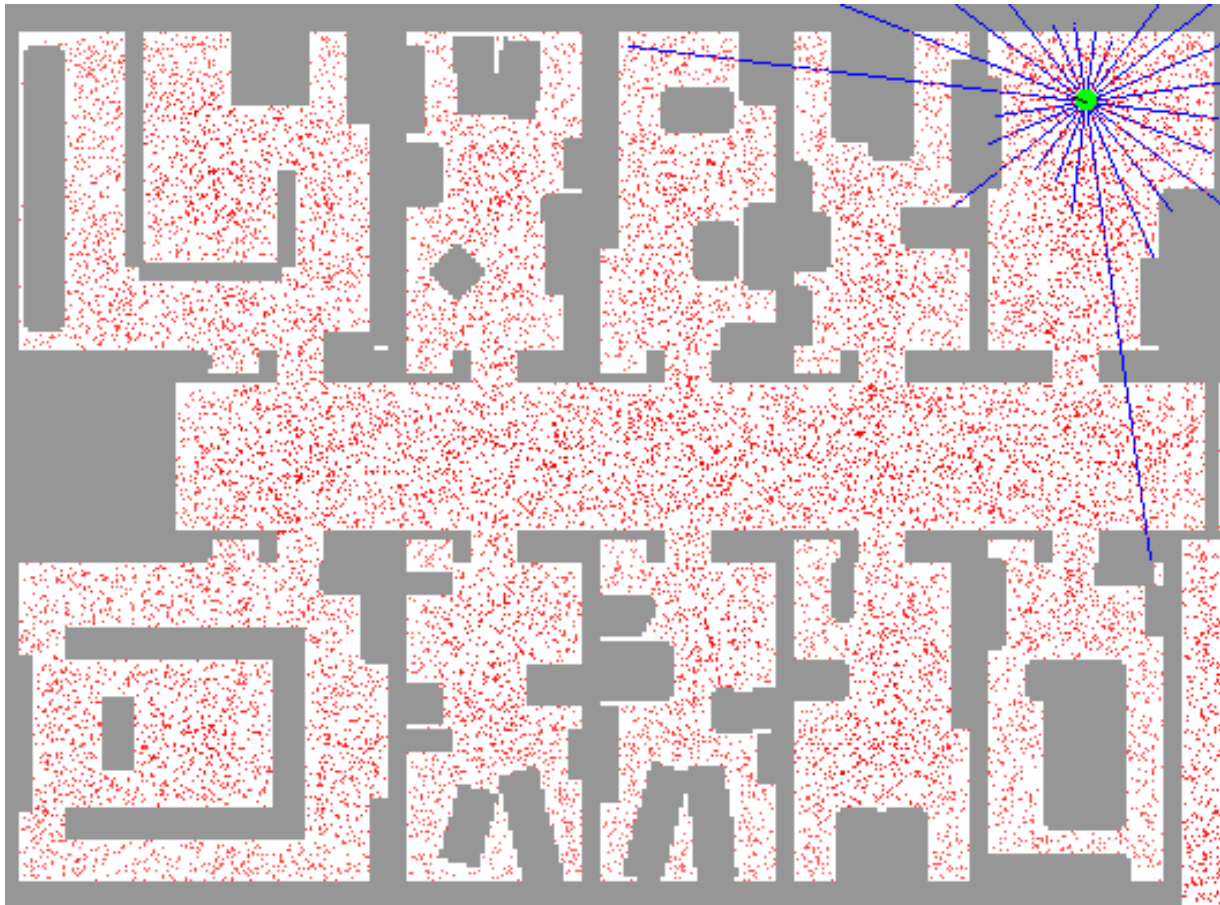


Sonar sensor



More Realistic Example Revisited

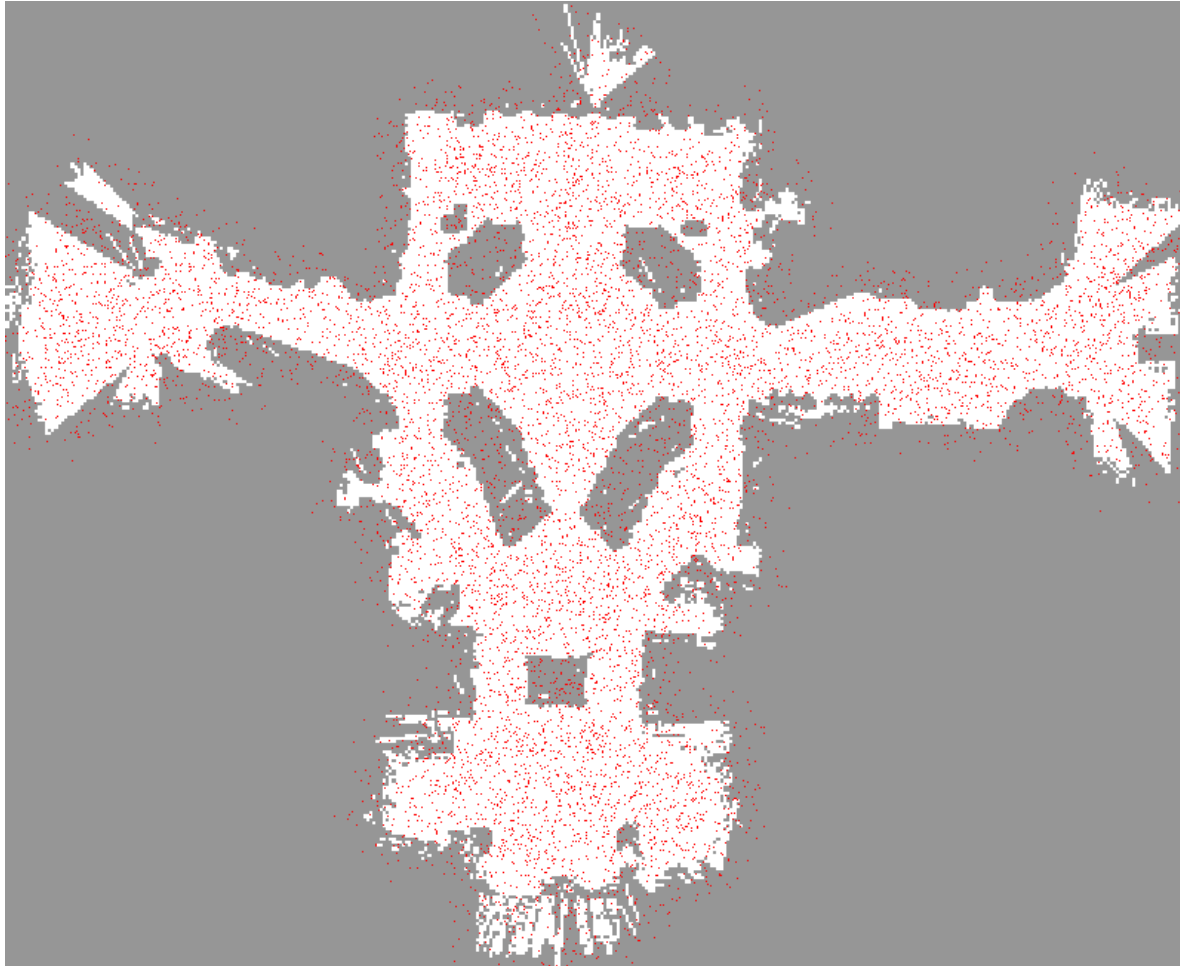
Range sensor based localization using Sonar

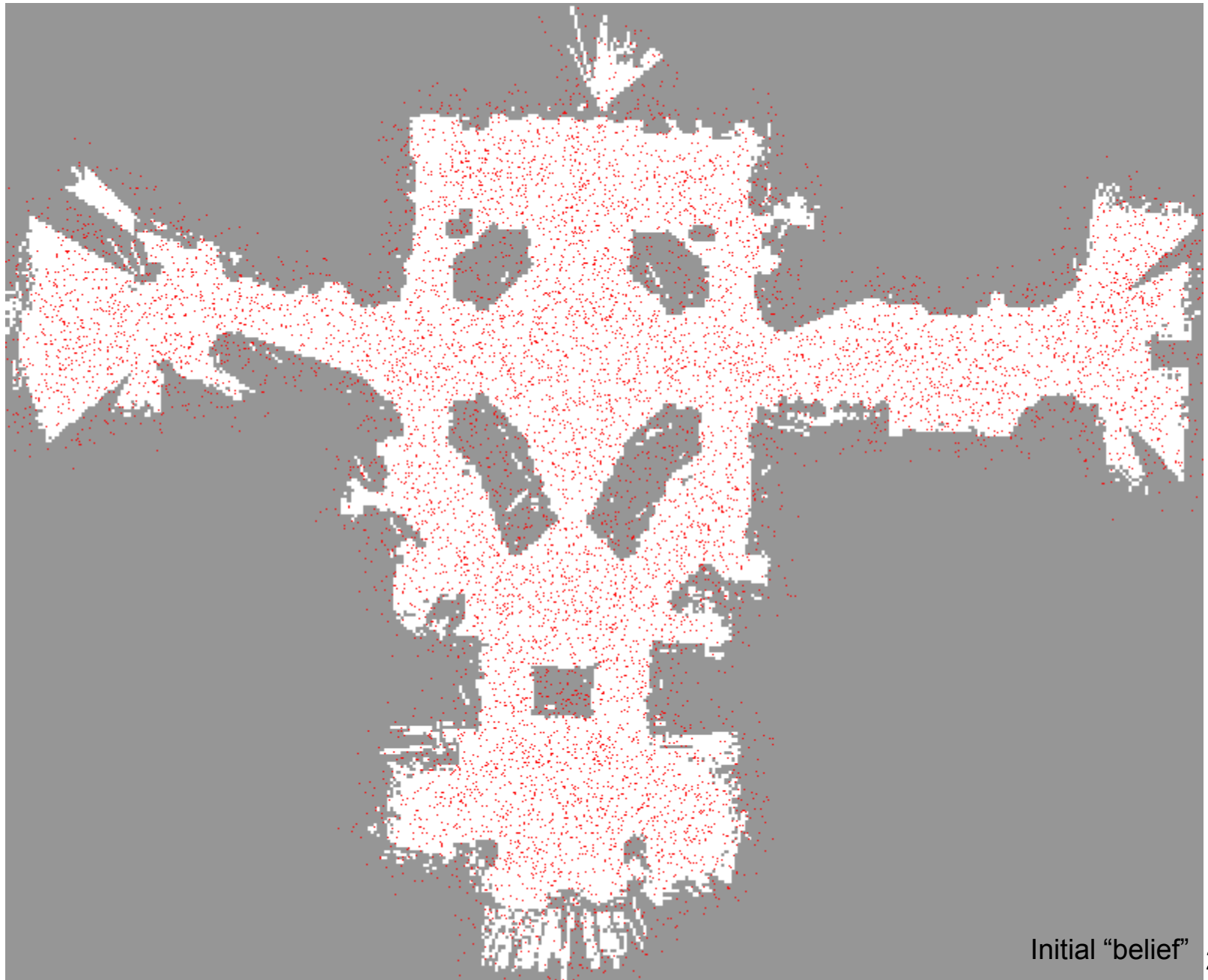


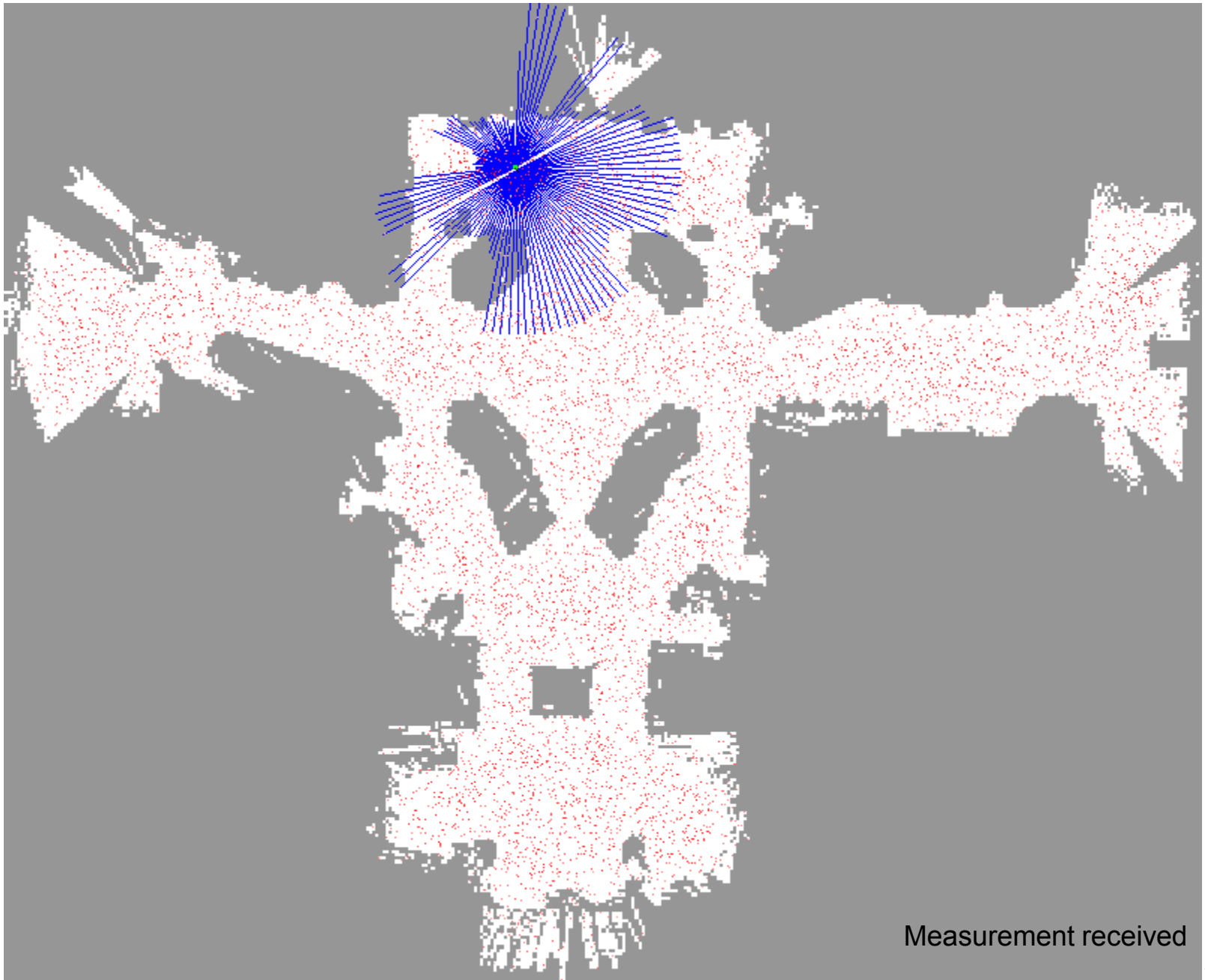


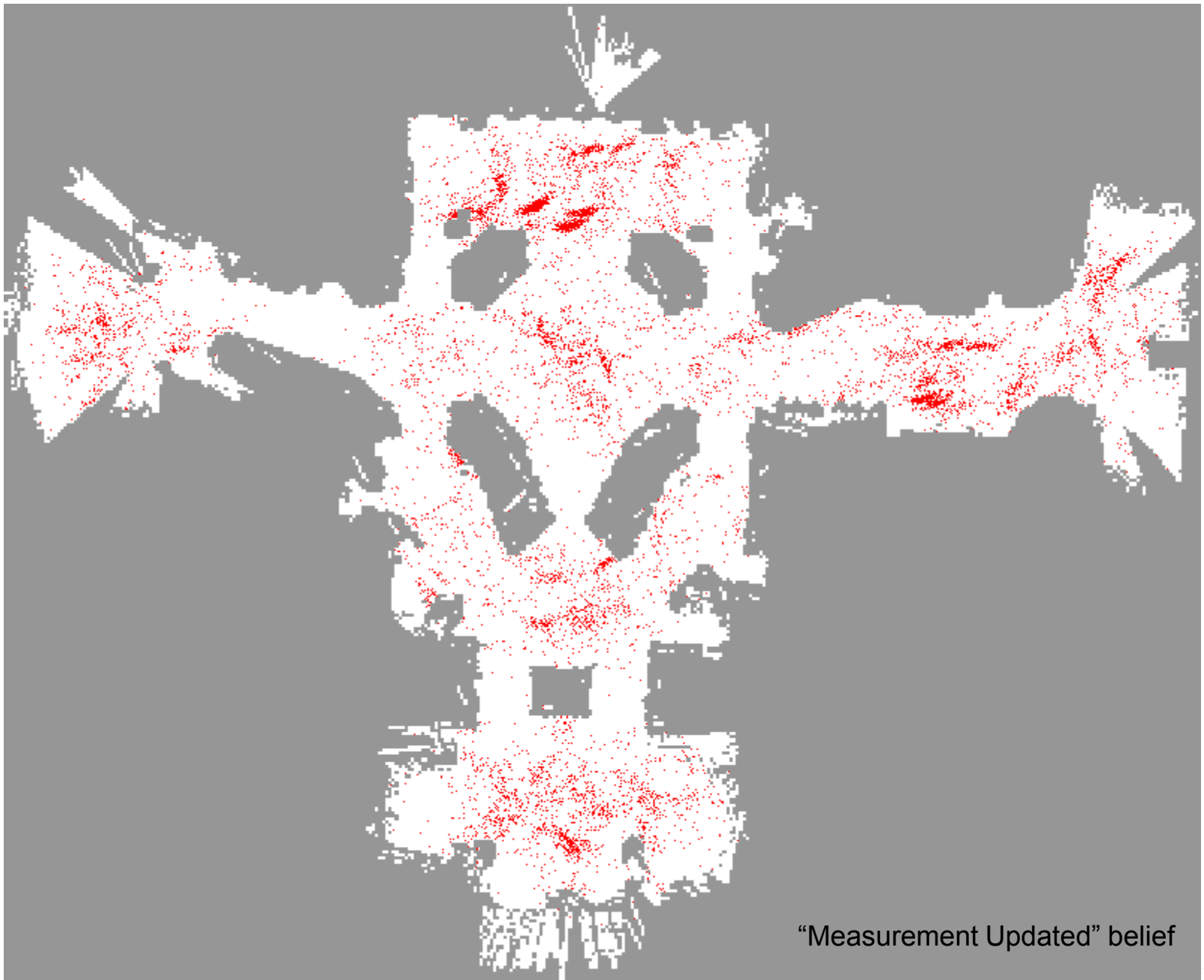
Another Example

Range sensor based localization using Radar in a Smithsonian Museum

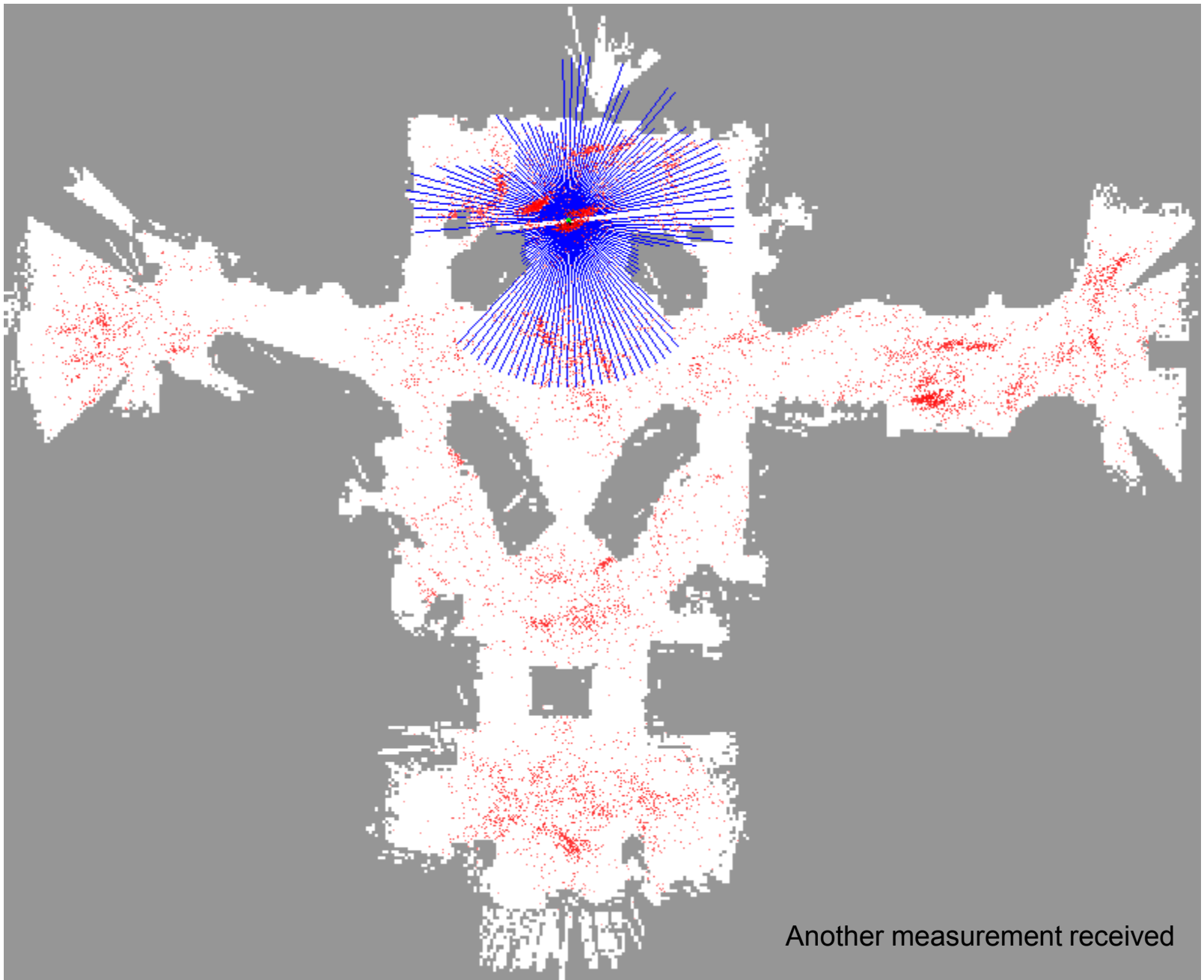


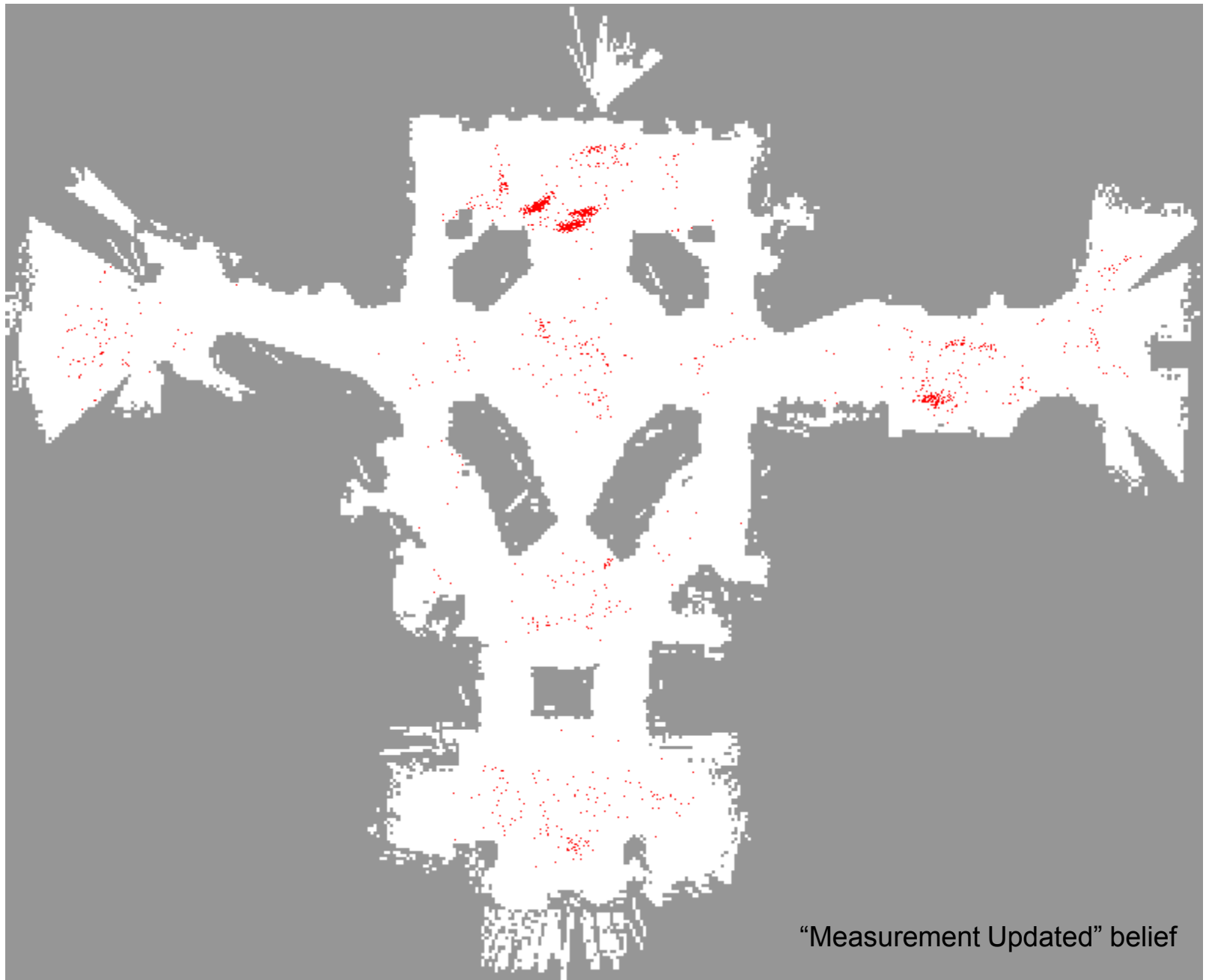


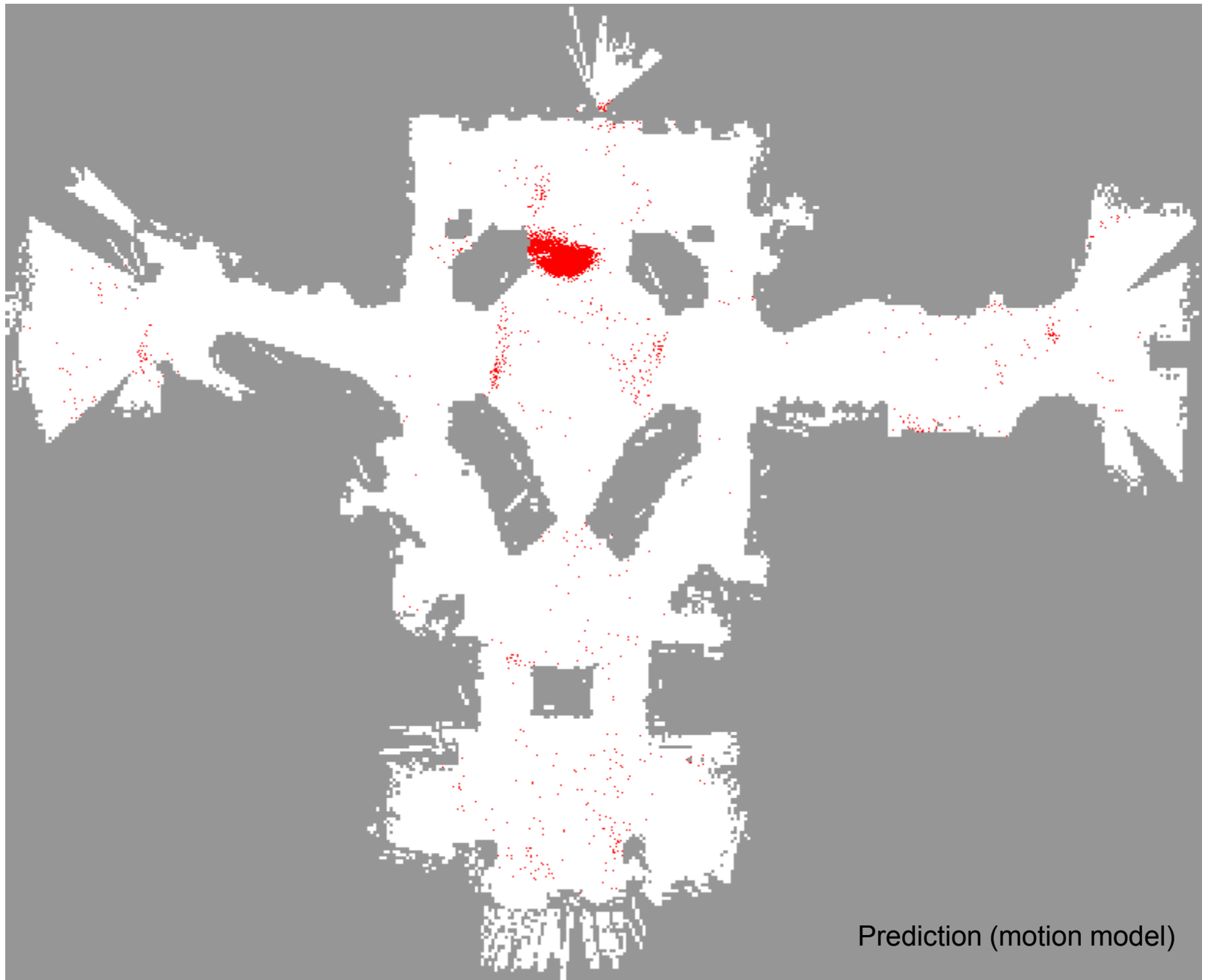




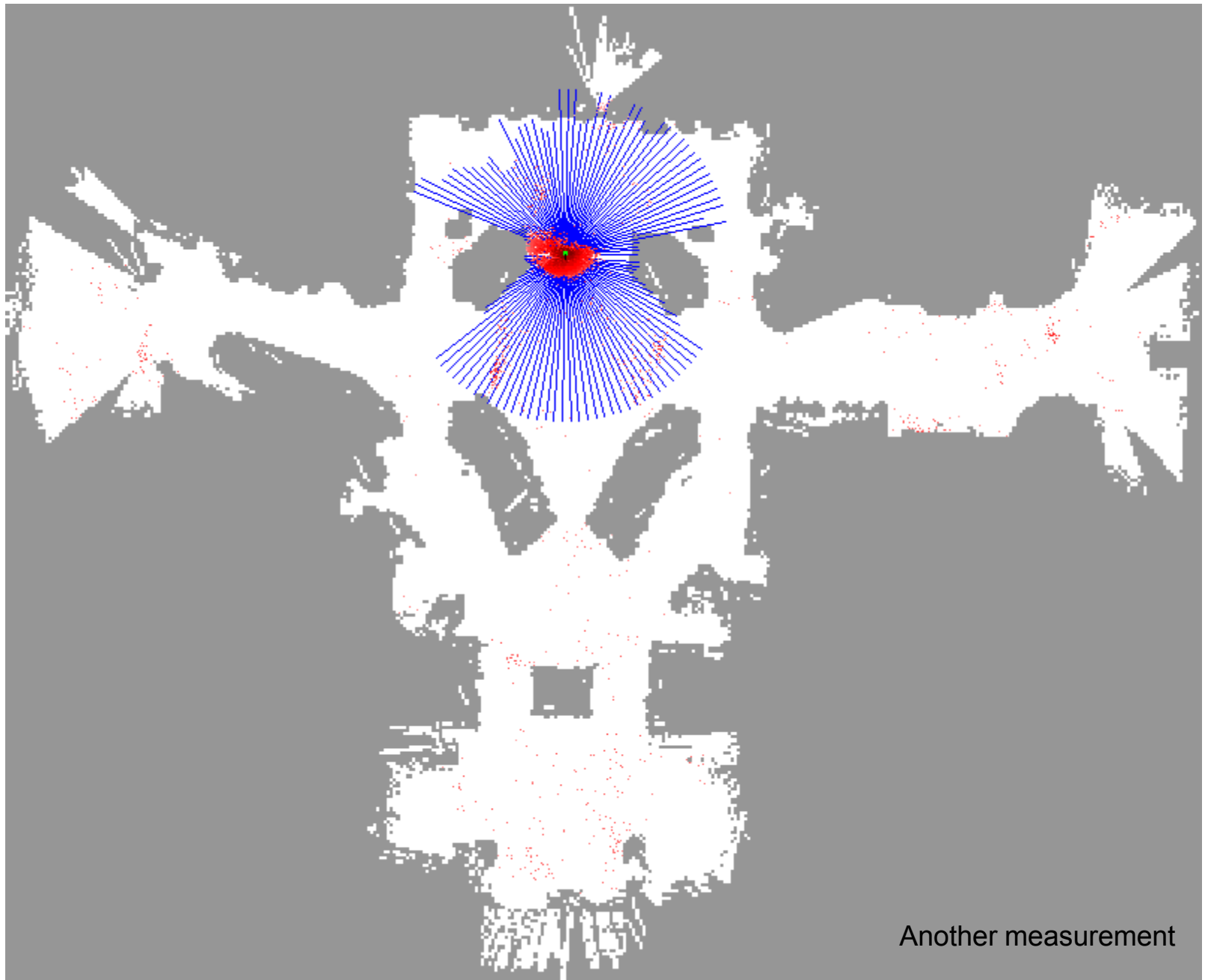
“Measurement Updated” belief



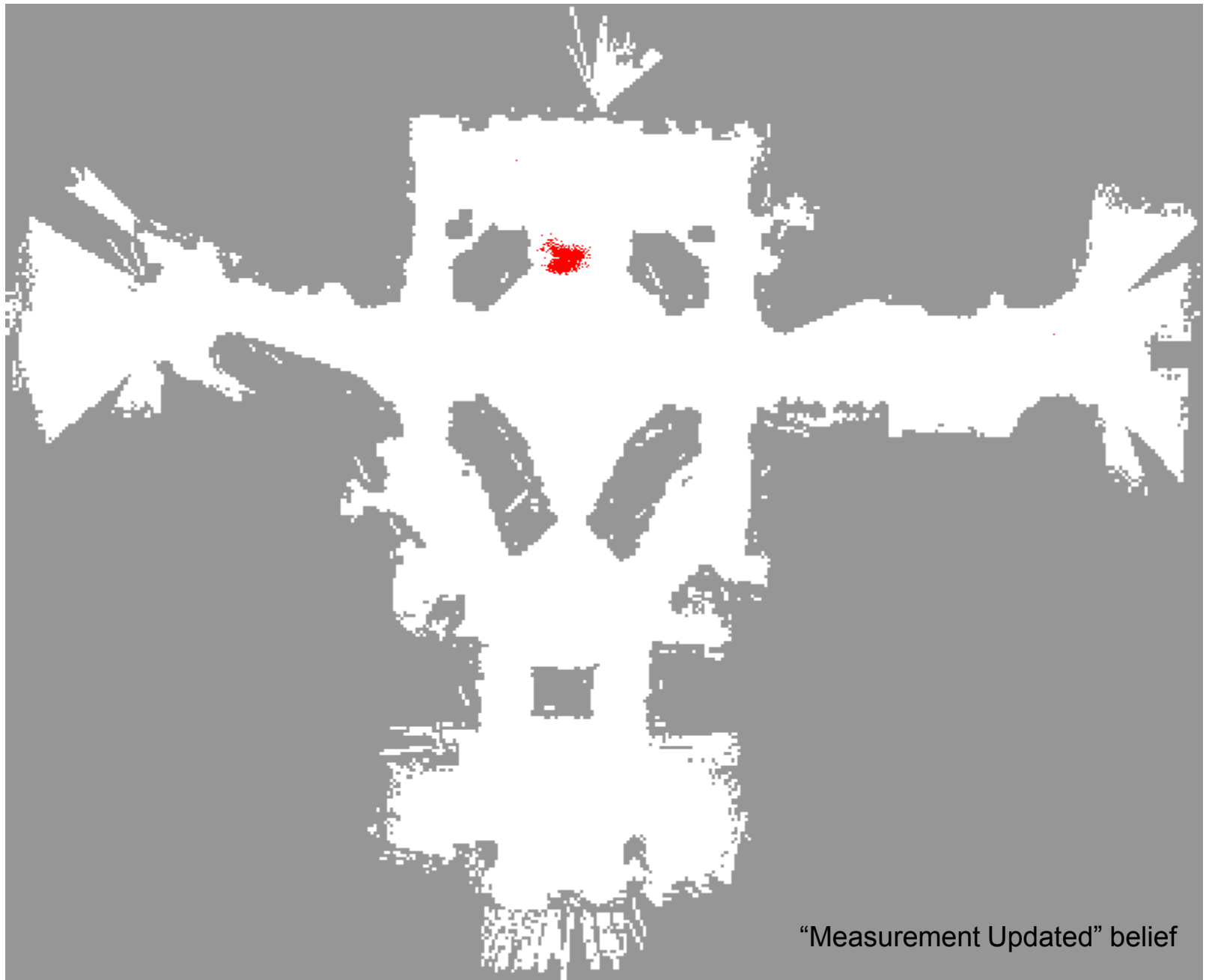


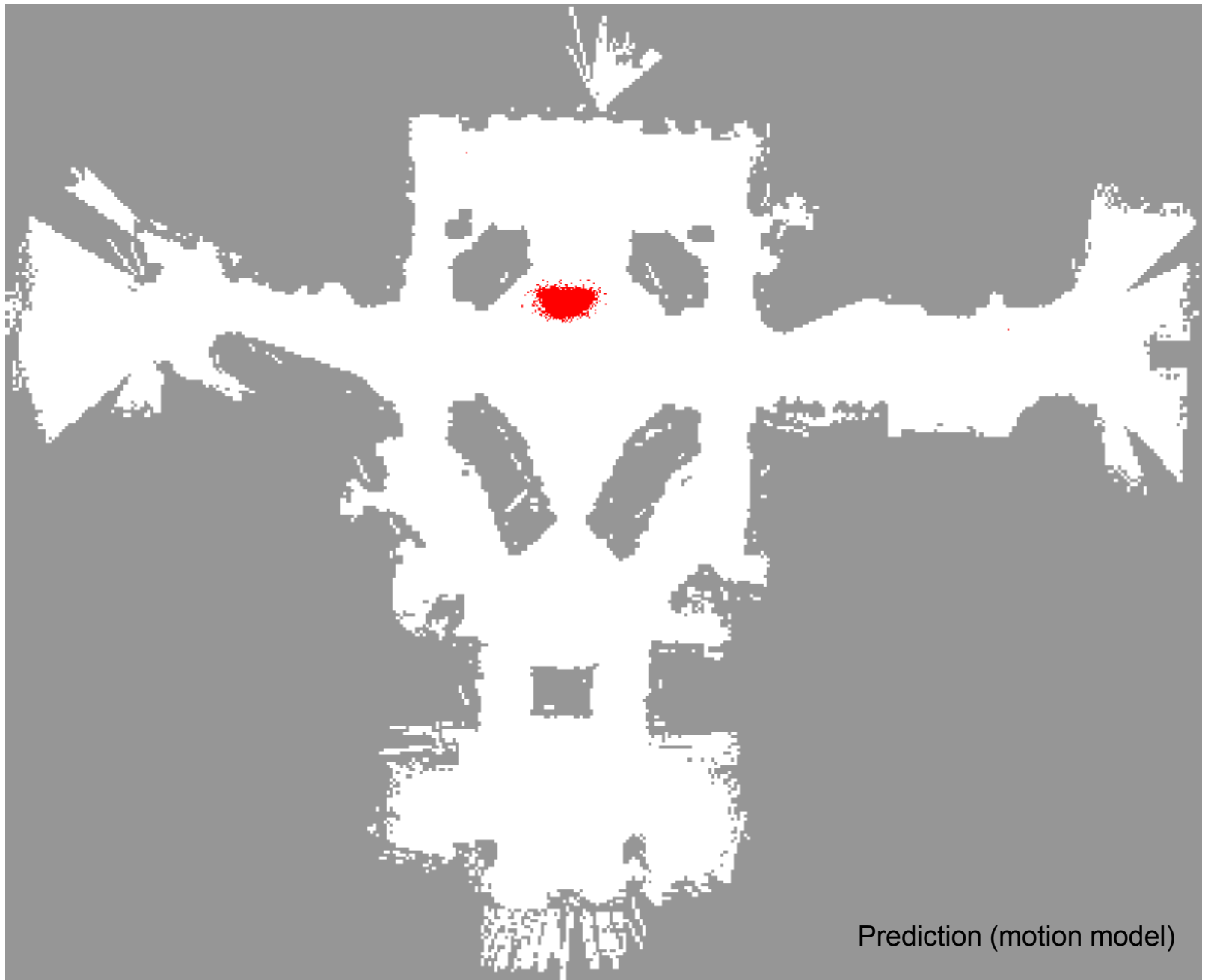


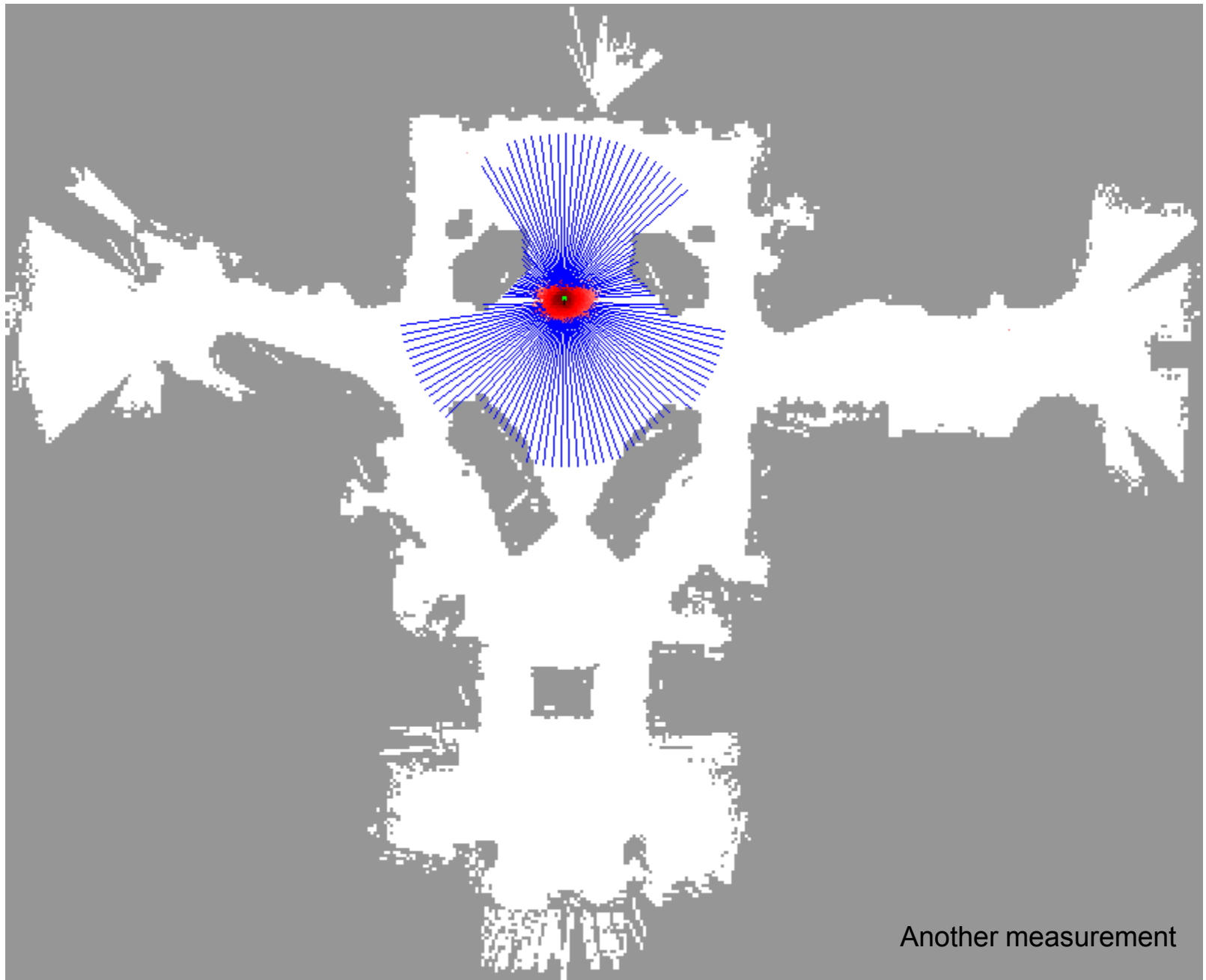
Prediction (motion model)

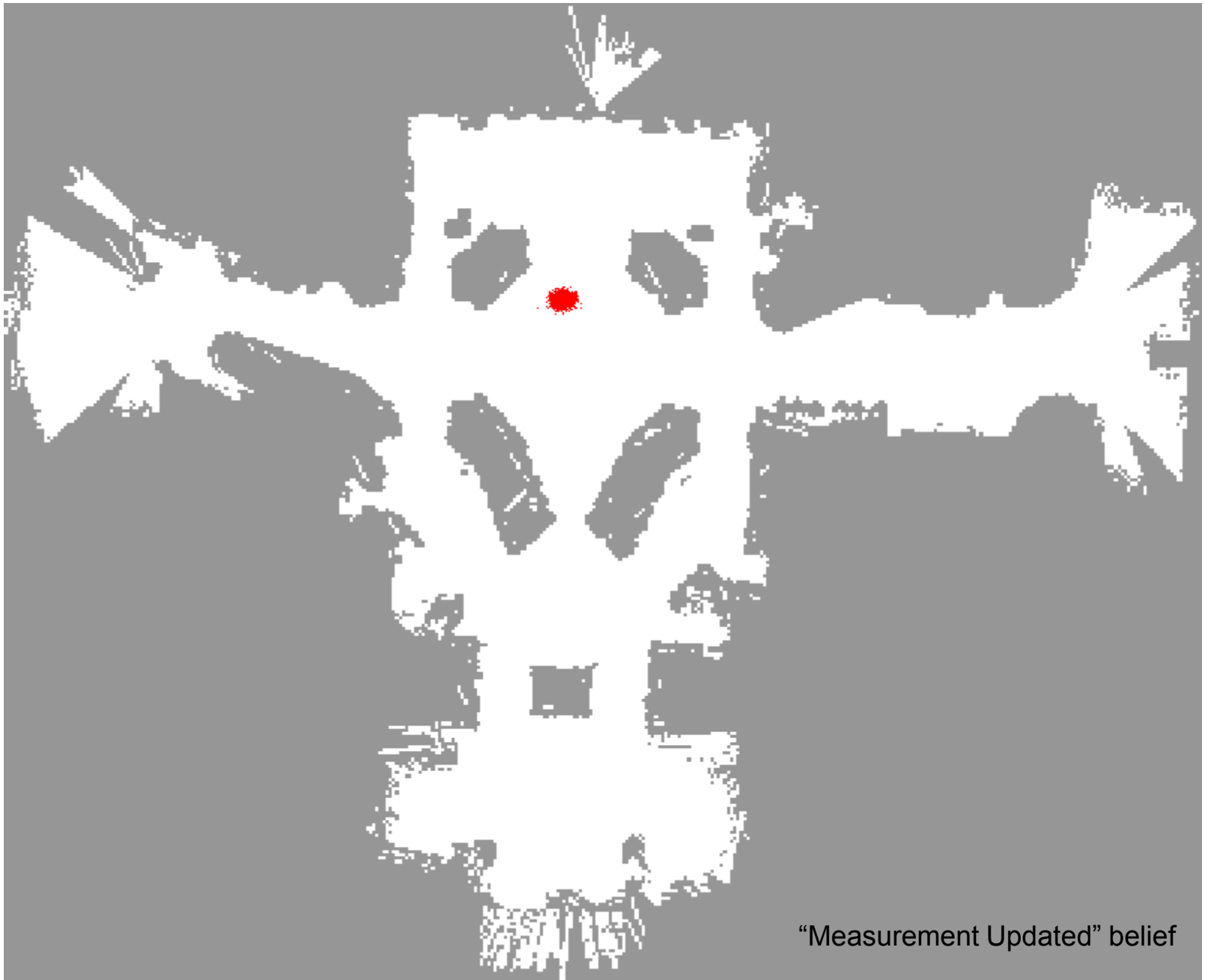


Another measurement

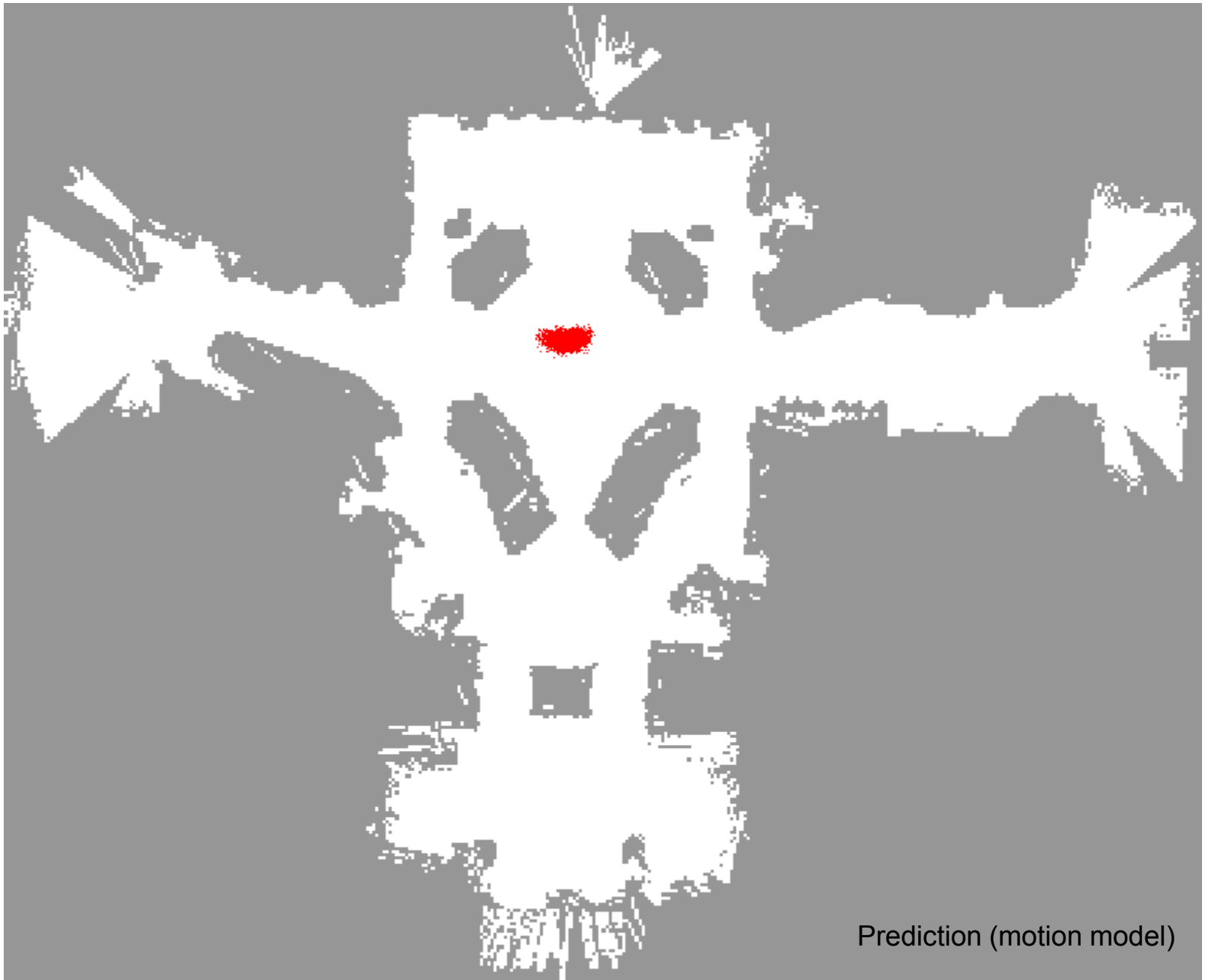


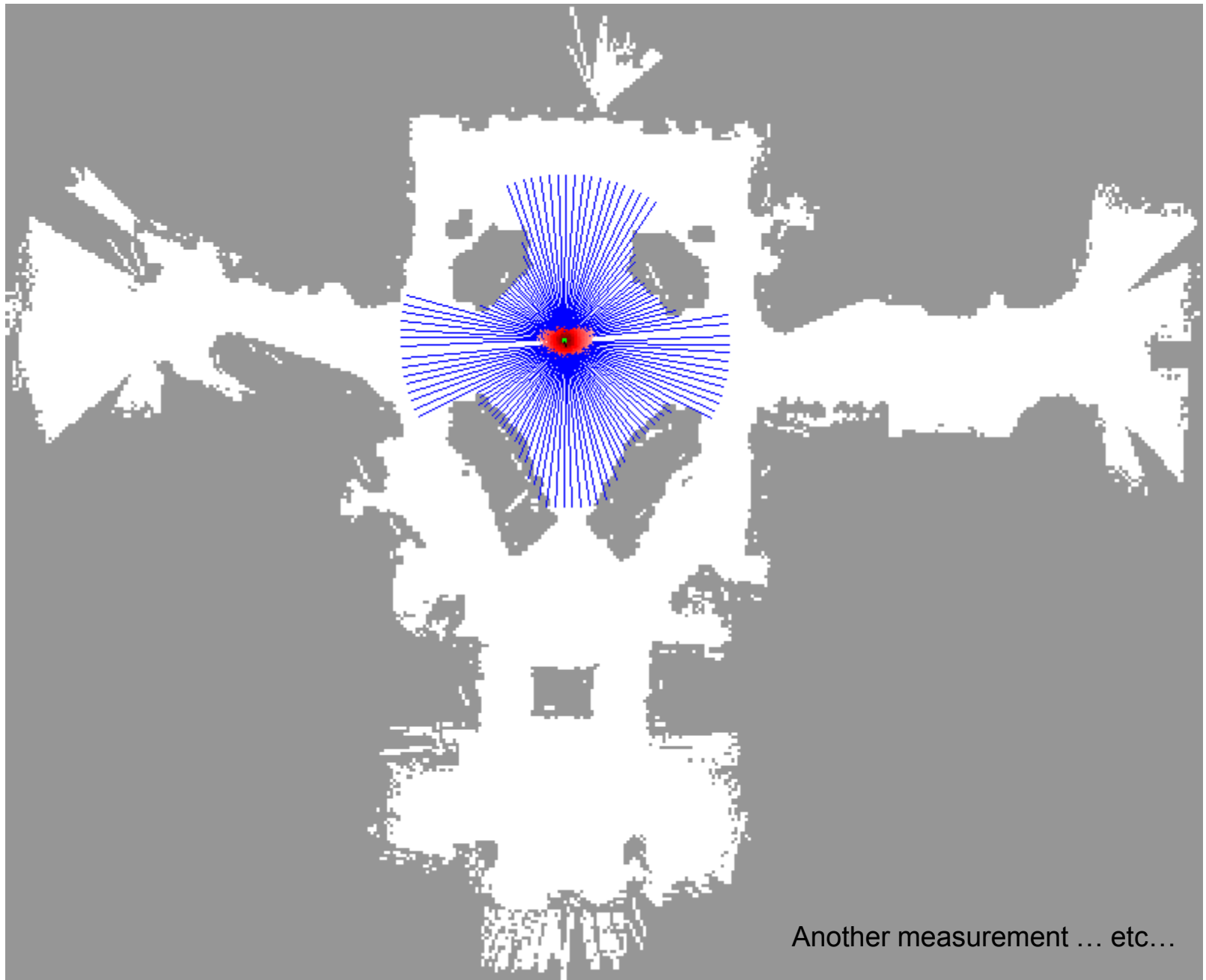






“Measurement Updated” belief





Another measurement ... etc...



Another Annotated Example

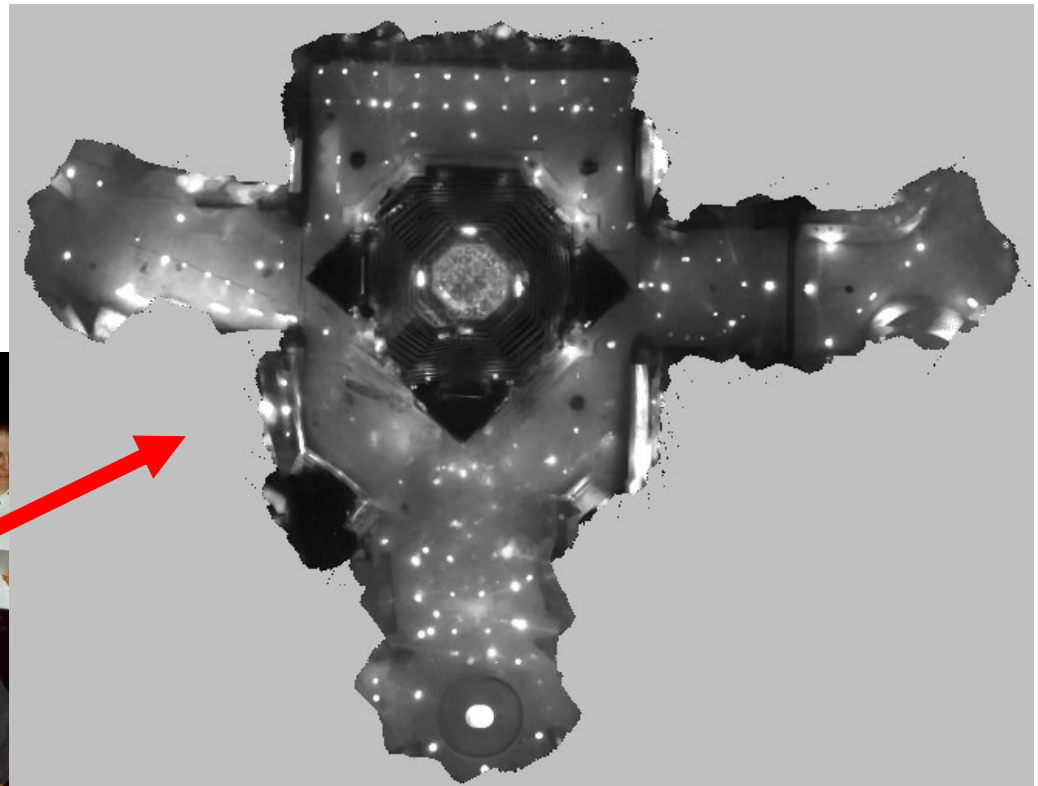
Comparison of Sonar and Laser Localization Performance





Localization with Ceiling Camera

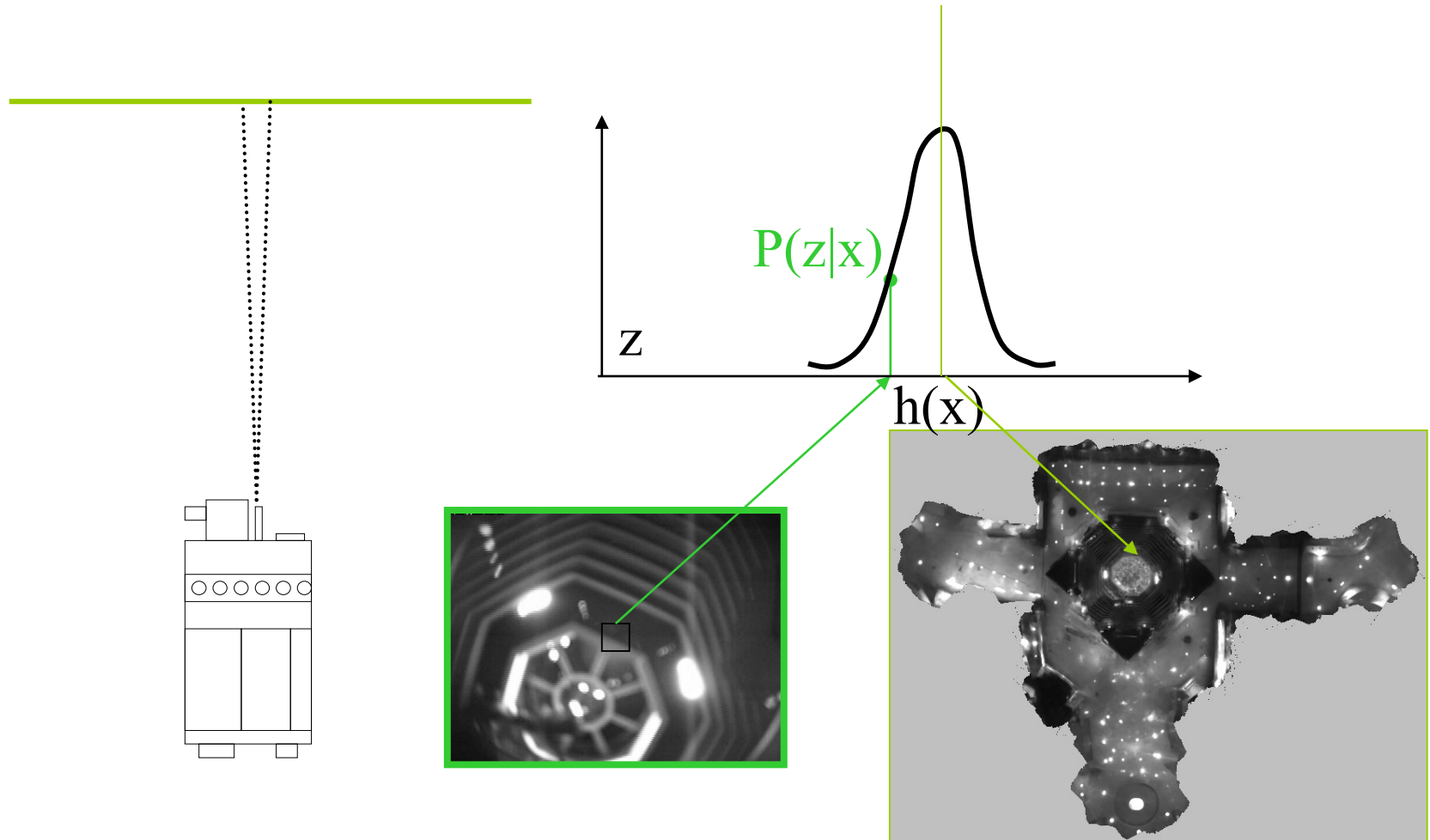
Yet another example where camera images are matched for measurement likelihood





Localization with Ceiling Camera

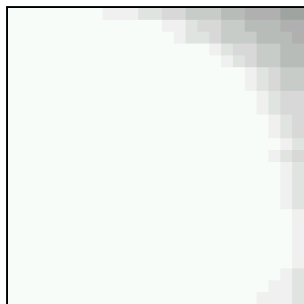
Yet another example where camera images are matched for measurement likelihood



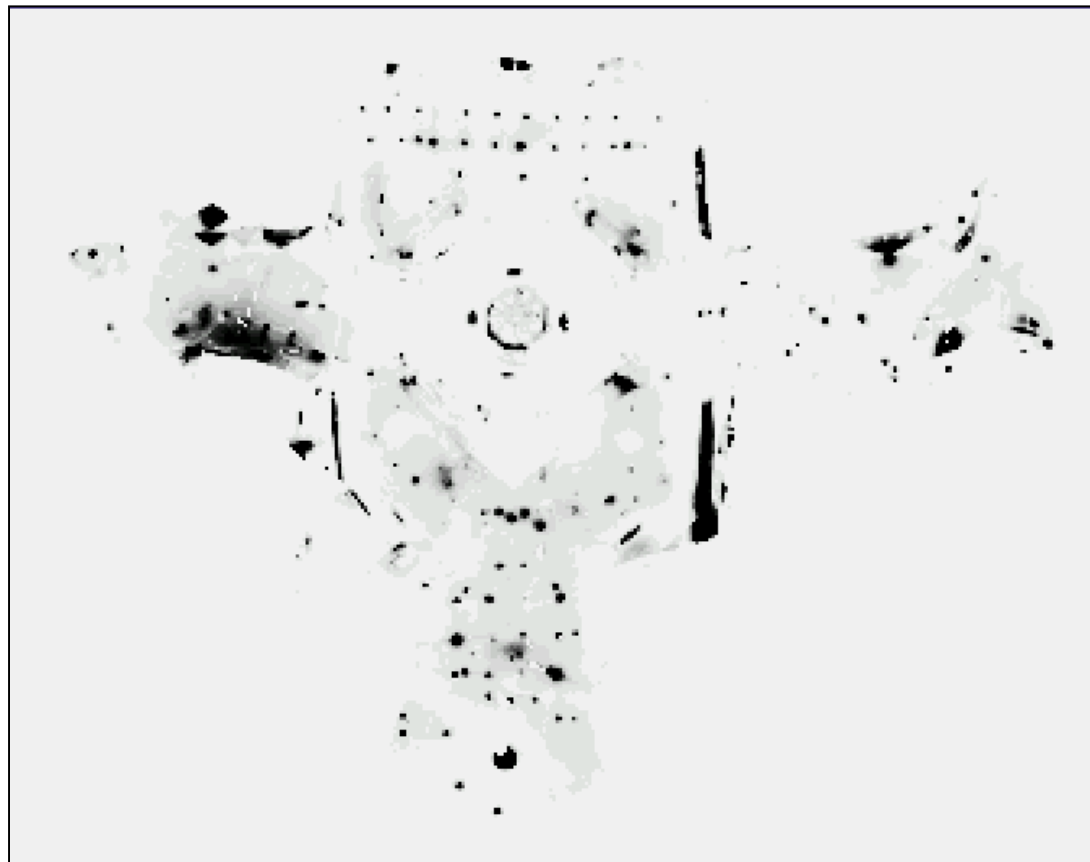


Camera: Under a Light

Measurement z :



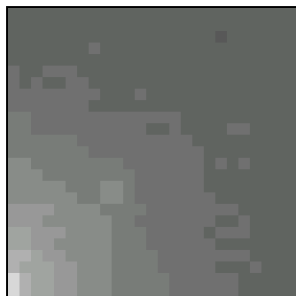
$P(z|x)$:



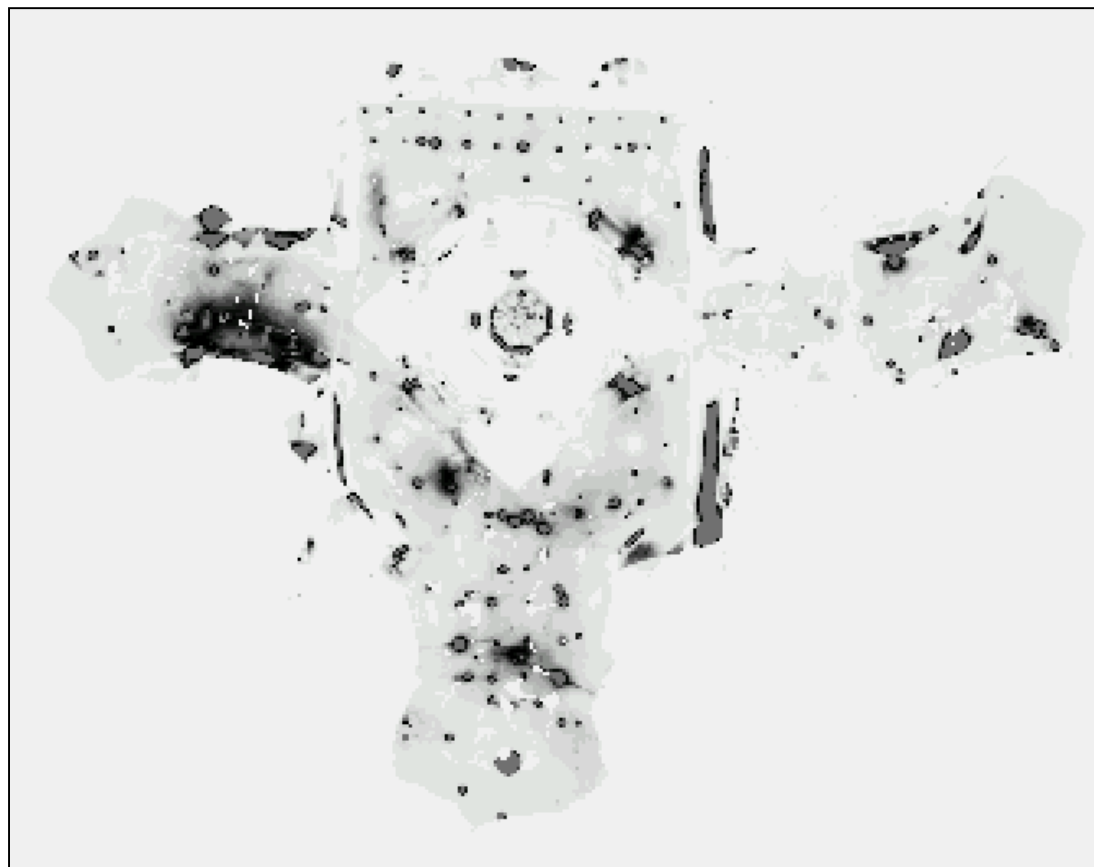


Camera: Next to a Light

Measurement z :

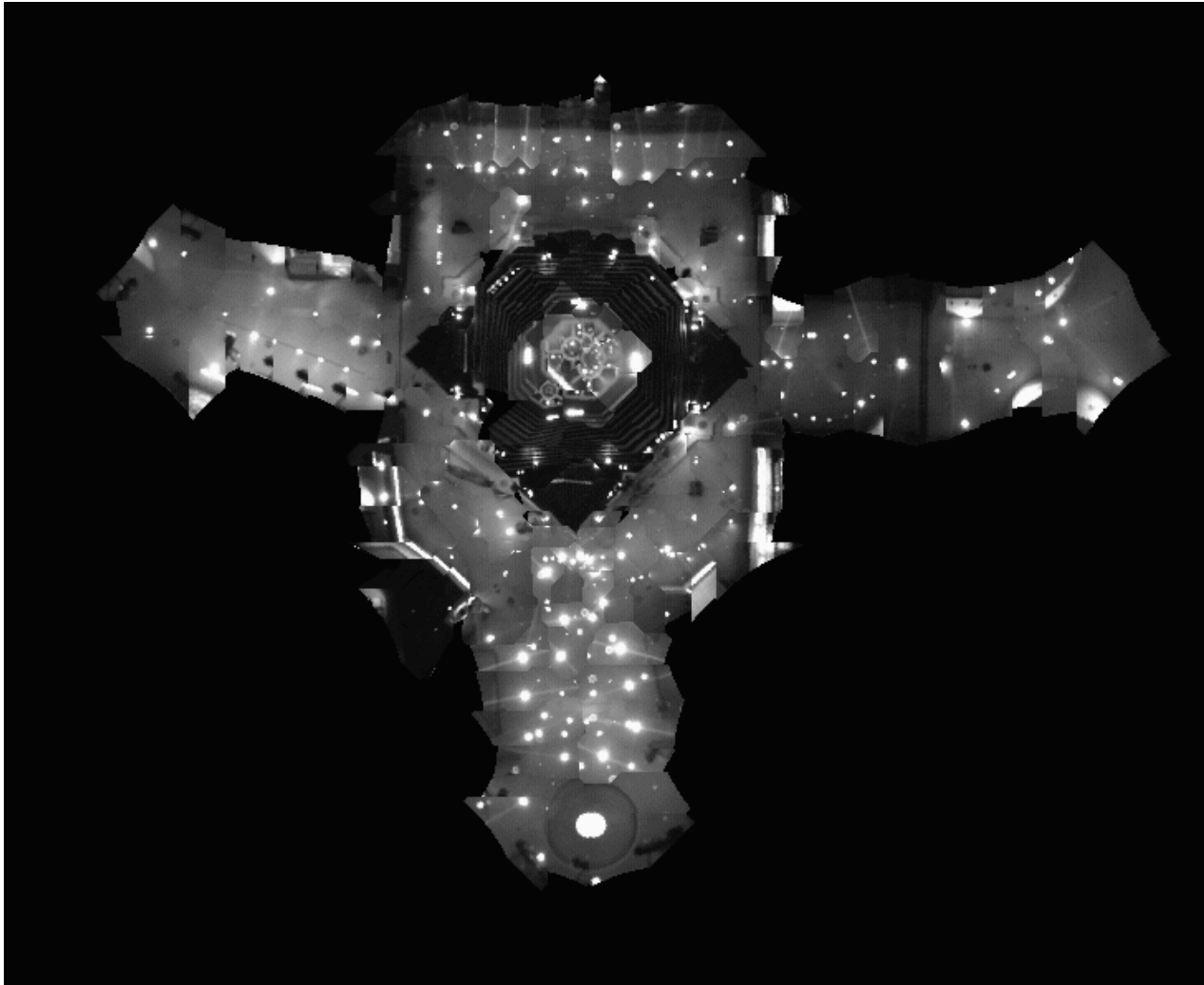


$P(z|x)$:





Ceiling Camera Global Localization





Recovery from Failures

- Plain MCL can solve global localization but not the *kidnapped robot problem*.
- **If there are no samples left around true state, particle filter will converge to a wrong state estimate,**
- **A significant problem if not handled!**
- **Question: How can we prevent this?**

A simple heuristic works well:
Injection of Random Particles



Recovery from Failures

- At every step of MCL, inject a small amount of random particles,
- Can be justified within the framework of motion model: “**small probability that robot may be kidnapped to a random location**”
- Hence: *Let the motion model generate the random particles.*
- ***Interesting (And Important) Questions:***
 - *How many random samples?*
 - *From which distribution?*



“Augmented MCL”

- How many random particles?
- **One intelligent way is to adapt number of random particles added based on an estimate of localization performance.**
- NOTE: Total number of particles is still the same!
- To implement this idea:
 - Monitor an approximate likelihood of Measurement,
 - Keep a long term (slow) and short term (fast) average of this likelihood,
 - Add more random samples in proportion to the amount short term likelihood falls below long term likelihood...

$$\frac{1}{M} \sum_{m=1}^M w_t^{[m]} \approx p(z_t \mid z_{1:t-1}, u_{1:t}, m)$$

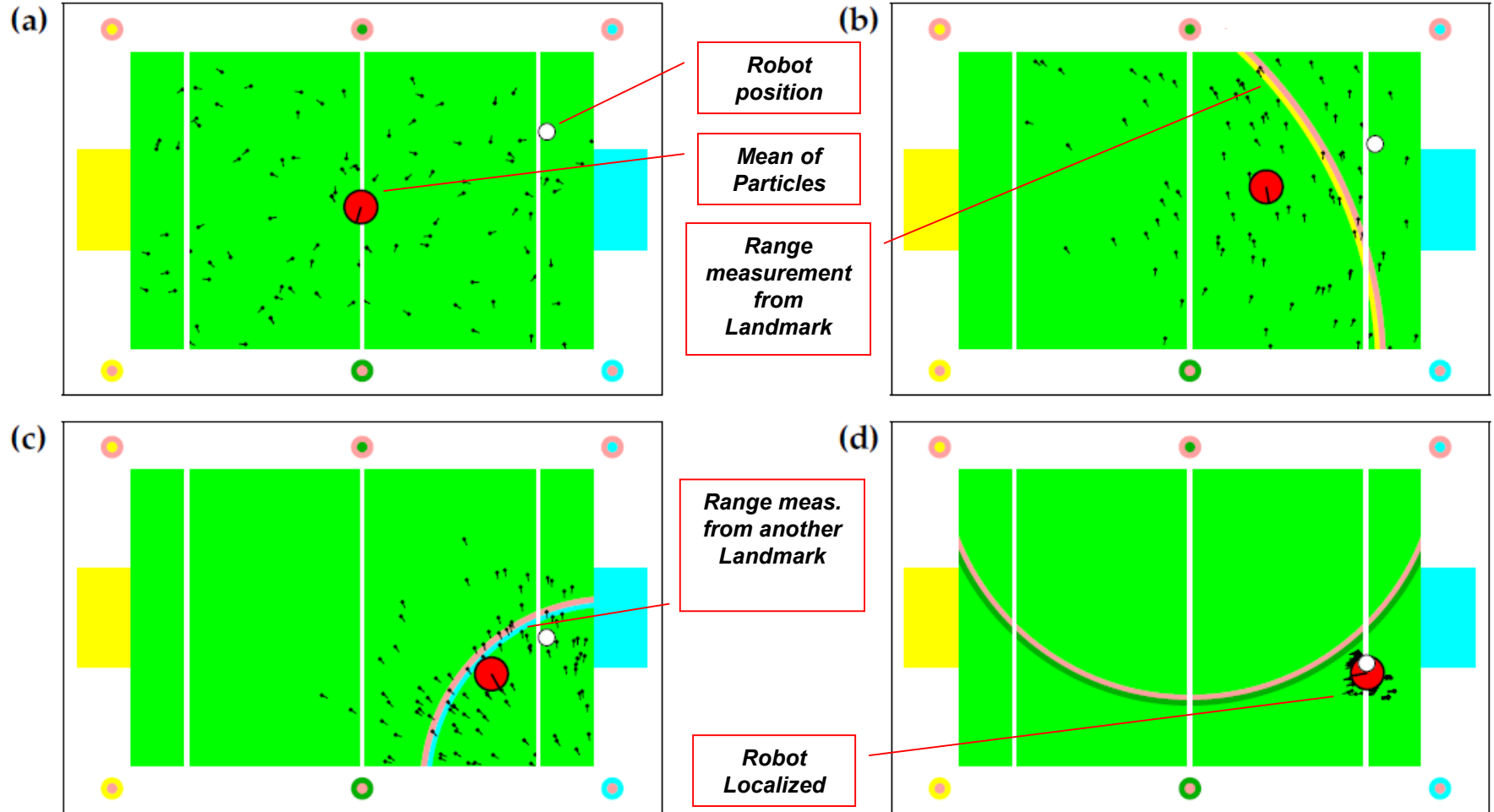


“Augmented MCL”

```
1:   Algorithm Augmented_MCL( $\mathcal{X}_{t-1}, u_t, z_t, m$ ):
2:     static  $w_{\text{slow}}, w_{\text{fast}}$ 
3:      $\bar{\mathcal{X}}_t = \mathcal{X}_t = \emptyset$ 
4:     for  $m = 1$  to  $M$  do
5:        $x_t^{[m]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[m]})$ 
6:        $w_t^{[m]} = \text{measurement\_model}(z_t, x_t^{[m]}, m)$ 
7:        $\bar{\mathcal{X}}_t = \bar{\mathcal{X}}_t + \langle x_t^{[m]}, w_t^{[m]} \rangle$ 
8:        $w_{\text{avg}} = w_{\text{avg}} + \frac{1}{M} w_t^{[m]}$ 
9:     endfor
10:     $w_{\text{slow}} = w_{\text{slow}} + \alpha_{\text{slow}}(w_{\text{avg}} - w_{\text{slow}})$ 
11:     $w_{\text{fast}} = w_{\text{fast}} + \alpha_{\text{fast}}(w_{\text{avg}} - w_{\text{fast}})$ 
12:    for  $m = 1$  to  $M$  do
13:      with probability  $\max\{0.0, 1.0 - w_{\text{fast}}/w_{\text{slow}}\}$  do
14:        add random pose to  $\mathcal{X}_t$ 
15:      else
16:        draw  $i \in \{1, \dots, N\}$  with probability  $\propto w_t^{[i]}$ 
17:        add  $x_t^{[i]}$  to  $\mathcal{X}_t$ 
18:      endwith
19:    endfor
20:    return  $\mathcal{X}_t$ 
```

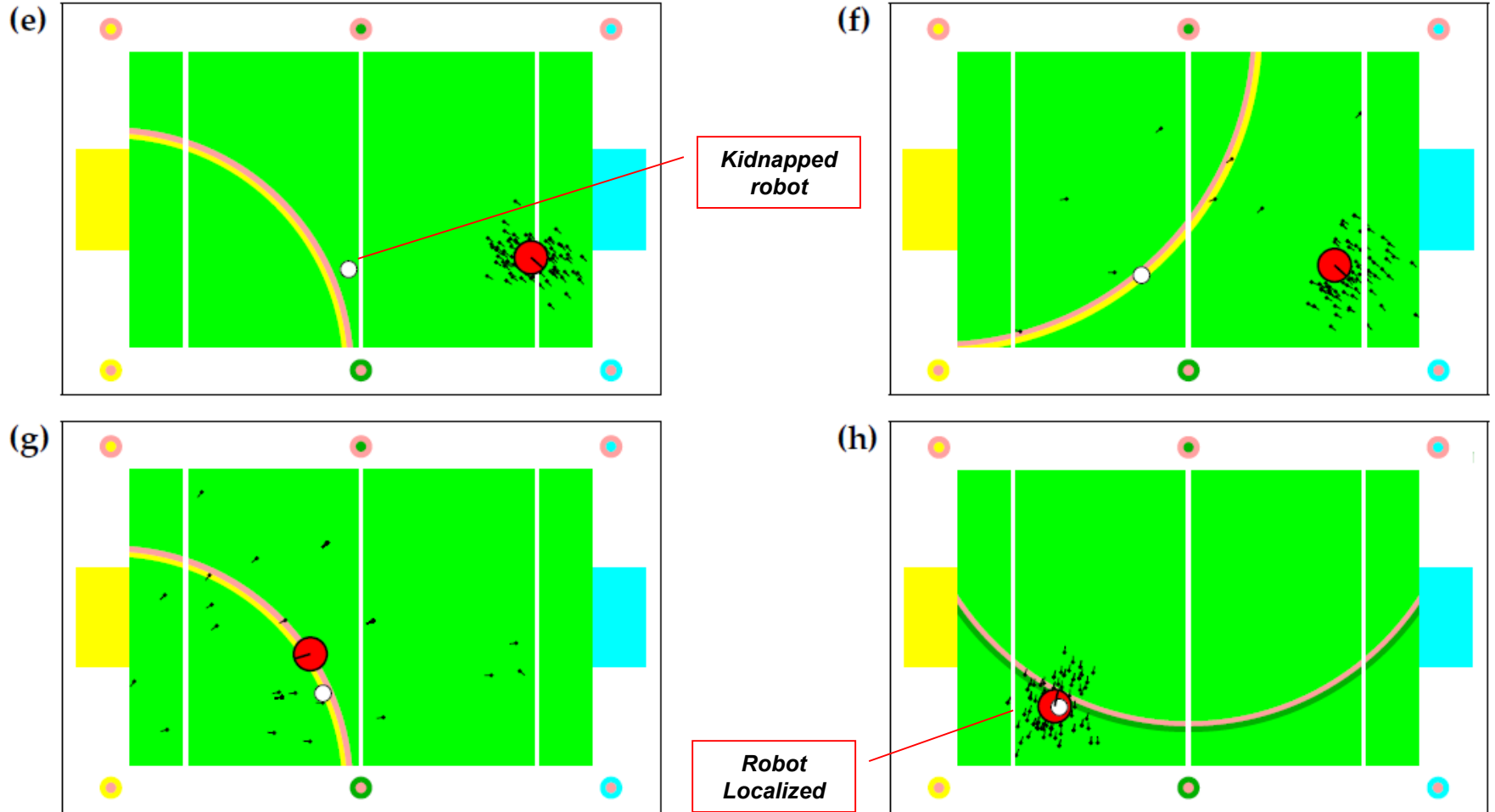


“Augmented MCL” Example



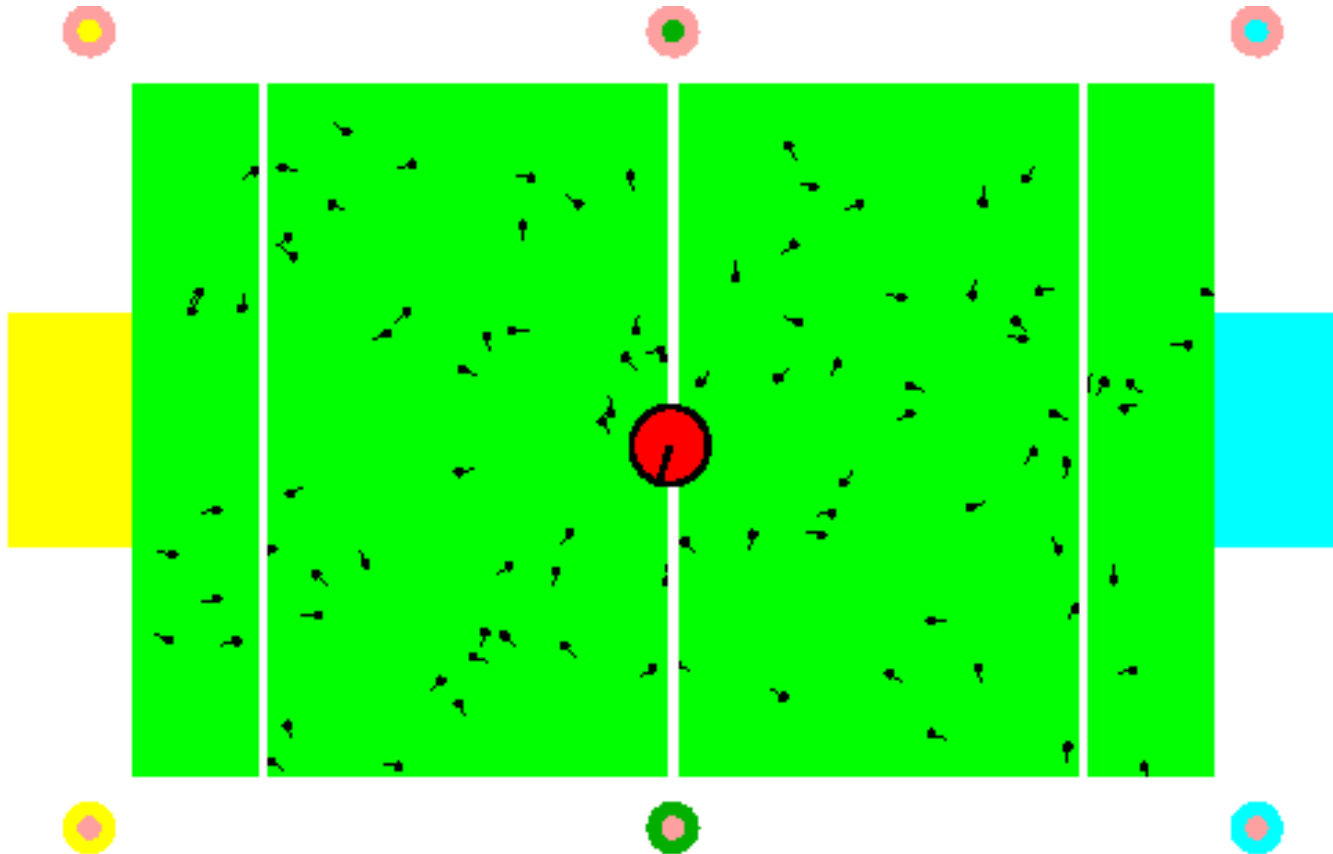


“Augmented MCL” Example





“Augmented MCL” Example





Mixture MCL

- Another surprising failure mode of MCL is with sensors with no noise.
- In this case, the approximate nature of the algorithm leads to the failure of MCL algorithm.
- One simple approach to partially solve this problem is to increase the amount of noise in the sensor model.
- Another approach is called “Mixture MCL”,
- Proposes to “reverse” the roles to the *motion* and *sensor models* for a small fraction (e.g. 5%) of particles. (Generate from sensor model, weight from motion model)

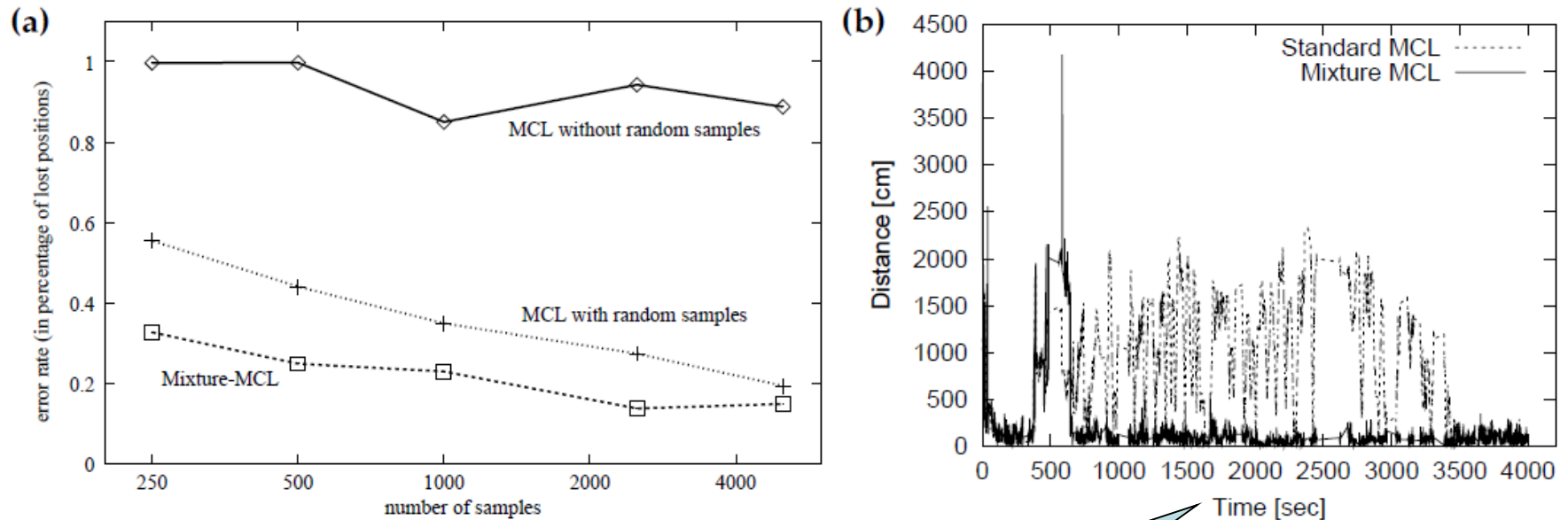


Mixture MCL

- Another surprising failure mode of MCL is with sensors with no noise.
- In this case, the approximate nature of the algorithm leads to the failure of MCL algorithm.
- One simple approach to partially solve this problem is to increase the amount of noise in the sensor model.
- Another approach is called “Mixture MCL”,
- Proposed “Mixture MCL” algorithm has “very good performance but difficult to implement”



Performance of “Robust” MCL



**Error as a function of time for Standard and Mixture MCL
(Ceiling Camera Localization Example)**



Adaptive Particle Set Size

- Upto now: Fixed M .
- Important parameter for efficiency and performance,
- Unfortunately M needs to be quite large (e.g. 100,000) for global localization problems to converge,
- Much lower numbers are required for tracking only once global localization is achieved.
- **We need a way to reduce M as global localization is transformed into a tracking problem.**
- **One algorithm:**
Kullback-Leibler Divergence (KLD)-Sampling



KLD Sampling

- One algorithm:
Kullback-Leibler Divergence (KLD)-Sampling
- *Kullback-Leibler Divergence* is a measure of distance/similarity between two density functions,
- The basic idea of KLD sampling is to determine M required to maintain a given similarity between the sample based approximation and true posterior density
- The state space is divided into a multi-dimensional grid of “bins” and number of bins K occupied by at least one sample is counted,
- Samples are generated until a KLD based bound is reached. The bound usually increases as K increases but saturates as samples concentrate.



KLD Sampling

```
1:  Algorithm KLD_Sampling_MCL( $\mathcal{X}_{t-1}, u_t, z_t, m, \varepsilon, \delta$ ):
2:       $\mathcal{X}_t = \emptyset$ 
3:       $M = 0, M_\chi = \infty, k = 0$ 
4:      for all  $b$  in  $H$  do
5:           $b = \text{empty}$ 
6:      endfor
7:      do
8:          draw  $i$  with probability  $\propto w_{t-1}^{[i]}$ 
9:           $x_t^{[M]} = \text{sample\_motion\_model}(u_t, x_{t-1}^{[i]})$ 
10:          $w_t^{[M]} = \text{measurement\_model}(z_t, x_t^{[M]}, m)$ 
11:          $\mathcal{X}_t = \mathcal{X}_t + \langle x_t^{[M]}, w_t^{[M]} \rangle$ 
12:         if  $(x_t^{[M]})$  falls into empty bin  $b$  then
13:              $k = k + 1$ 
14:              $b = \text{non-empty}$ 
15:             if  $(k > 1)$  then
16:                  $M_\chi := \frac{k-1}{2\varepsilon} \left\{ 1 - \frac{2}{9(k-1)} + \sqrt{\frac{2}{9(k-1)}} z_{1-\delta} \right\}^3$ 
17:             endif
18:              $M = M + 1$ 
19:             while  $(M < M_\chi)$ 
20:                 return  $\mathcal{X}_t$ 
```

Initialize the set of "bins" to empty

Update the number of non-empty bins

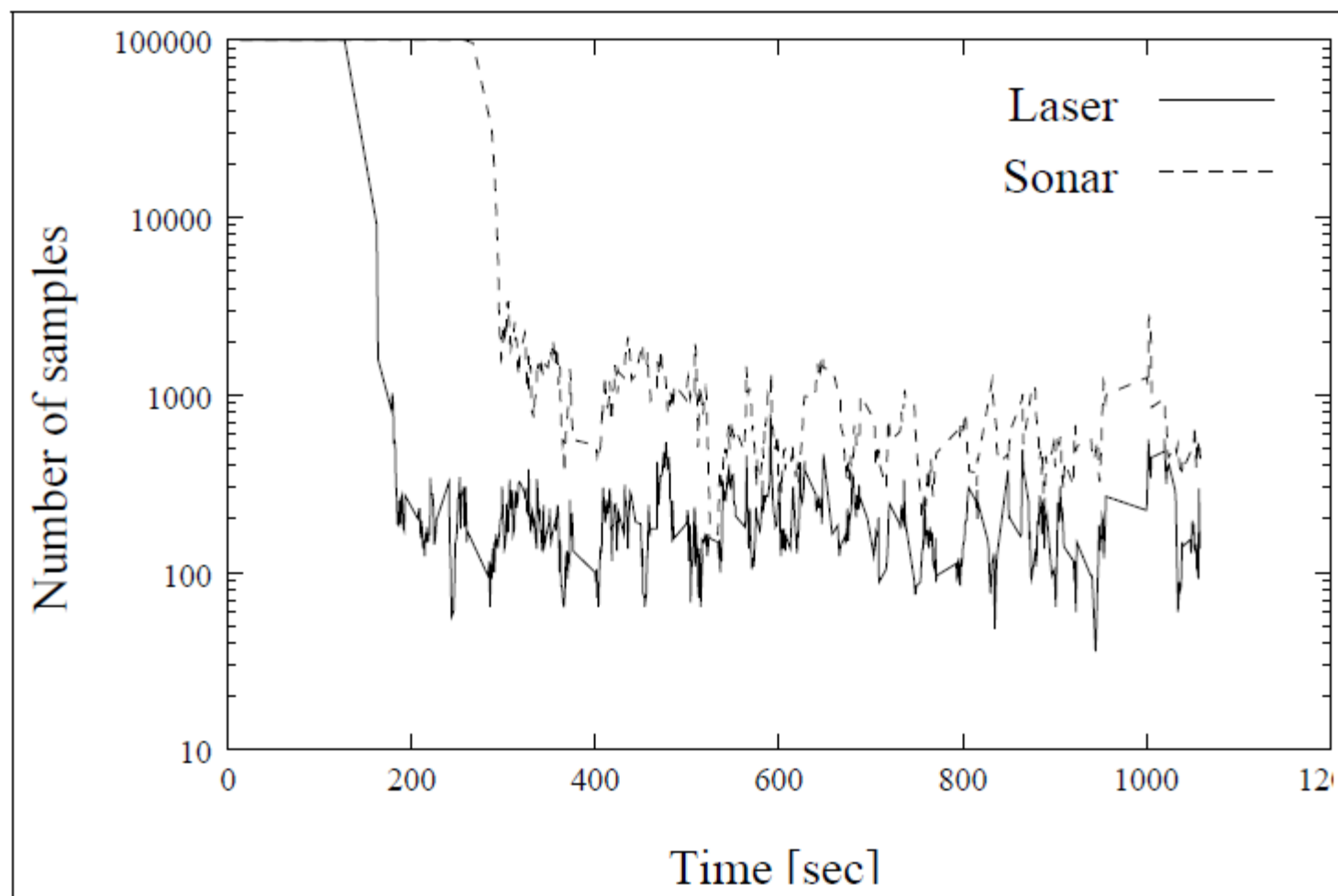
Requires KLD closeness params: With probability $1-\delta$, the mismatch between two densities wanted to be less than ε .

This algorithm accepts and returns samples with associated weights
-
No resampling

Continue adding samples until the bound is exceeded



KLD Sampling





Localization in Dynamic Environments





Localization in Dynamic Environments

- Two primary approaches:
- **State Augmentation**
 - Add all hidden states to the estimation process,
 - State space dimension increases very quickly,
 - Usually difficult for Particle Filters,
- **Outlier Rejection**
 - Make a decision on the “source” of the measurement,
 - Reject the measurement if it is coming from an unexpected obstacle with high likelihood
 - Very similar reasoning to the EM Learning we have seen for the sensor model



A Comparison of Localization

	EKF	MHT	Coarse (topological) grid	fine (metric) grid	MCL
Measurements	landmarks	landmarks	landmarks	raw measurements	raw measurements
Measurement noise	Gaussian	Gaussian	any	any	any
Posterior	Gaussian	mixture of Gaussians	histogram	histogram	particles
Efficiency (memory)	++	++	+	-	+
Efficiency (time)	++	+	+	-	+
Ease of implementation	+	-	+	-	++
Resolution	++	++	-	+	+
Robustness	-	+	+	++	++
Global localization	no	yes	yes	yes	yes



Summary

- Grid based Localization using Histogram Filters, Resolution of the grid and its effects,
- Monte-carlo localization using Particle Filters, accuracy/complexity trade-off,
- Global localization, kidnapped robot problem,
- Augmented MCL,
- Mixture MCL,
- KLD-Sampling
- Dynamic Environments,
- State augmentation, sensor outlier rejection